

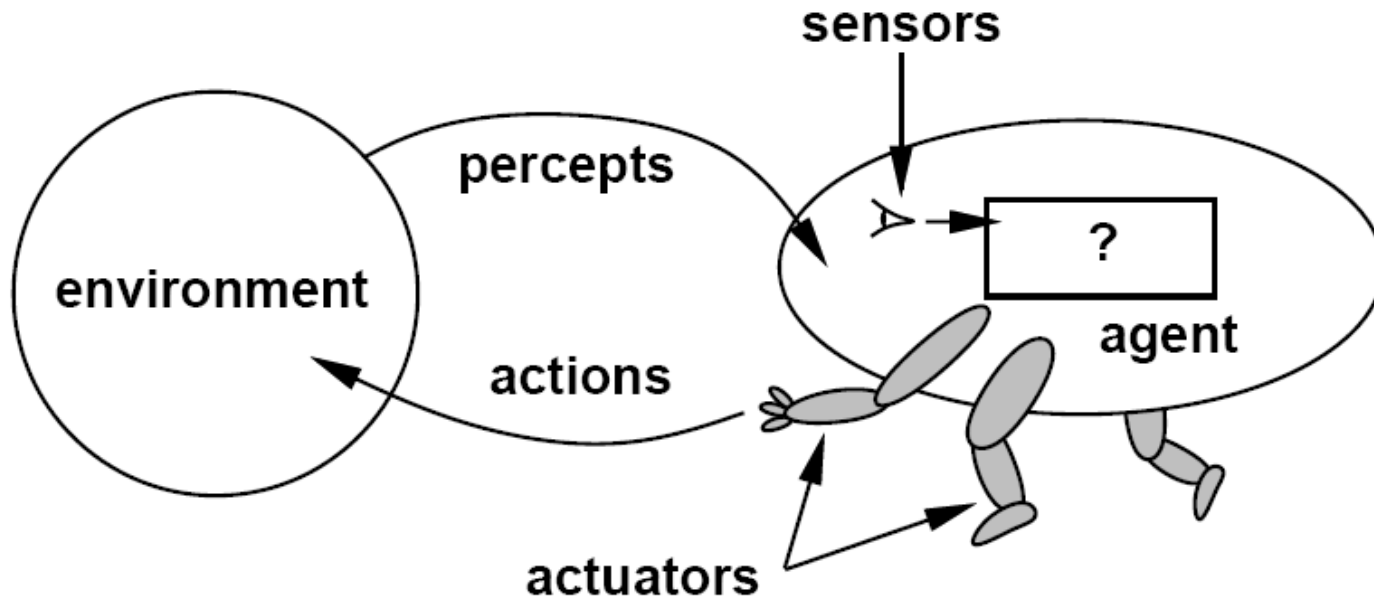
Search

Qian Wang

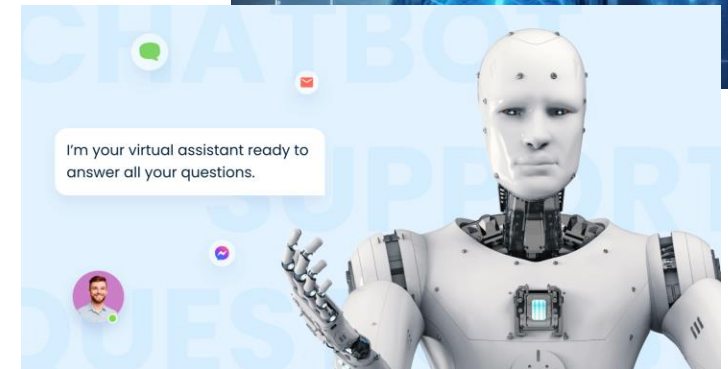
School of Biomedical Engineering
ShanghaiTech University

Agent

- An **agent** is any entity that can:
 - **Perceive** its environment through **sensors**;
 - **Act** upon that environment through **actuators**.

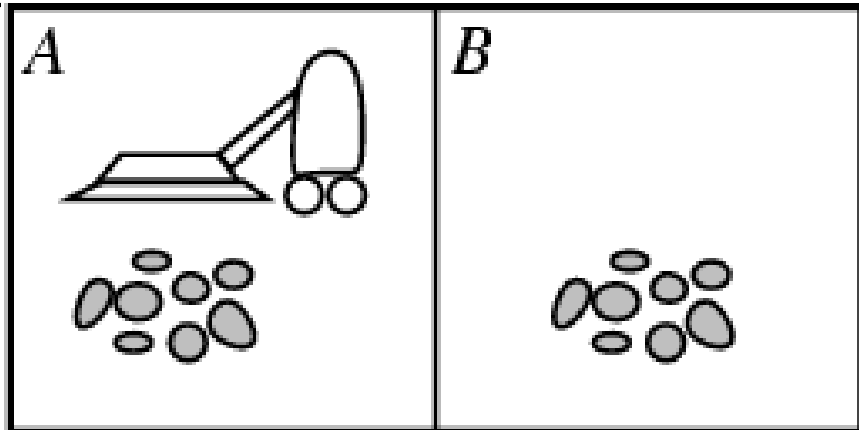


Agent = Perception + Action



P-E-A-S Framework

- To design and evaluate agents, we define:
 - **P**erformance Measure: What the agent is trying to optimize;
 - **E**nvironment: The external context it operates in;
 - **A**ctuators: Tools it uses to take action;
 - **S**ensors: Tools it uses to gather information.



A vacuum-cleaning agent

- Performance: Cleanliness, efficiency
- Environment: Room layout, dirt status
- Actuators: Wheels, cleaning brush
- Sensors: Location, dirt detector

Rationality

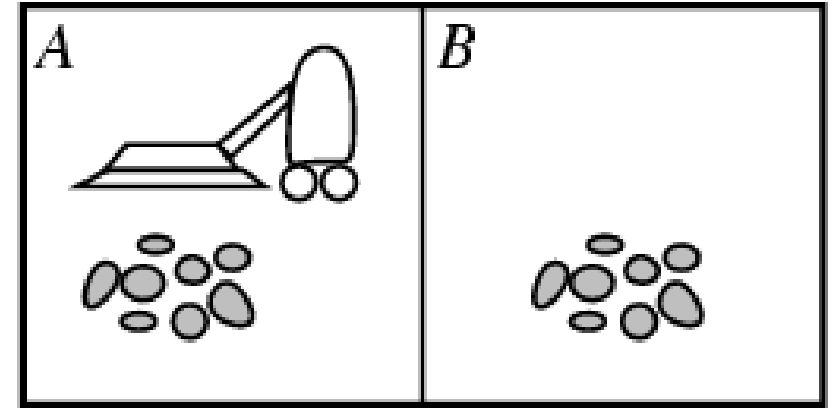
- For every possible percept sequence, a **rational agent** selects the action that is expected to **maximize its performance measure**, based on the evidence from its percepts and its built-in knowledge.
- The performance measure, often formalized as a **utility function**, serves as an objective criterion to evaluate the agent's behavior and determine how successful it is in achieving its goals.

Expected Utility (EU)

$$EU(\text{action}) = \sum_{\text{outcome}} P(\text{outcome}|\text{action}) \cdot U(\text{outcome})$$

Example: Cleaner-Agent

- Percepts:
 - [Location, Status] (e.g., [A, Dirty])
- Actions:
 - Left, Right, Clean, NoOp
- Utility Function:
 - Cleaner-Agent([location, status]) returns an **action**
 - *if status = Dirty then return Clean*
 - *else if location = A then return Right*
 - *else if location = B then return Left*



Environment Types

- Fully Observable vs. Partially Observable
- Static vs. Dynamic
- Discrete vs. Continuous
- Deterministic vs. Stochastic
- Single-agent vs. Multi-agent
- Episodic vs. Sequential

Fully Observable vs. Partially Observable

- Do the agent's sensors give it access to the complete state of the environment?
- For any given world state, are the values of all the variables known to the agent?



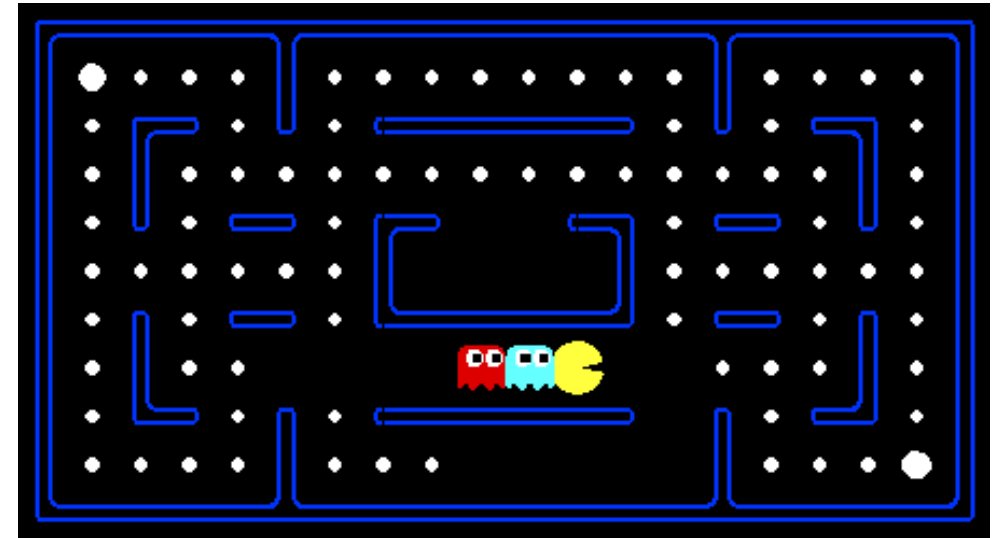
Deterministic vs. Stochastic

- Is the next state of the environment completely determined by the current state and the agent's action?
 - **Deterministic**: unique successor state given current state and action;
 - **Stochastic**: distribution over successor states given current state and action;
 - **Strategic**: the environment is deterministic except for the actions of other agents.



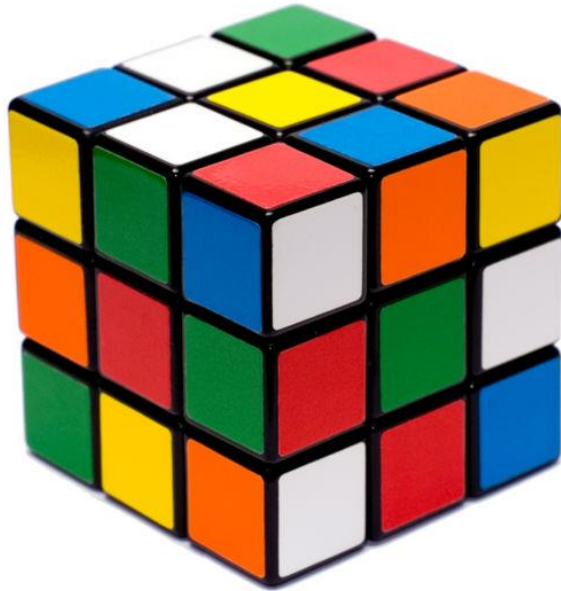
Episodic vs. Sequential

- Is the agent's experience divided into unconnected episodes, or is it a coherent sequence of observations and actions?



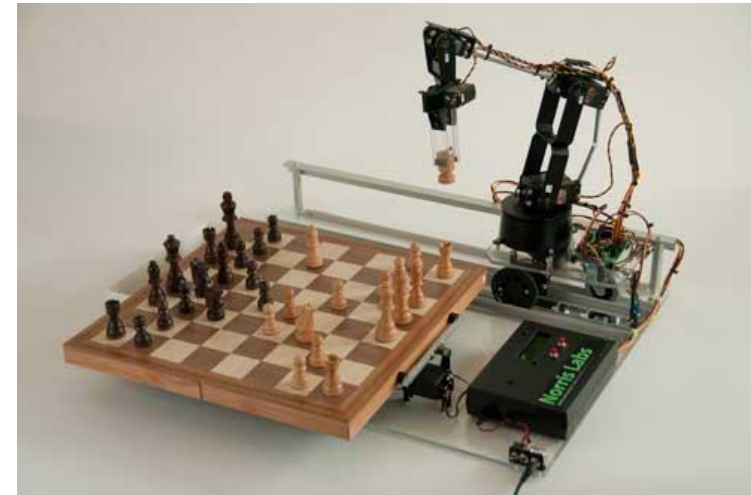
Static vs. Dynamic

- Is the world changing while the agent is thinking?



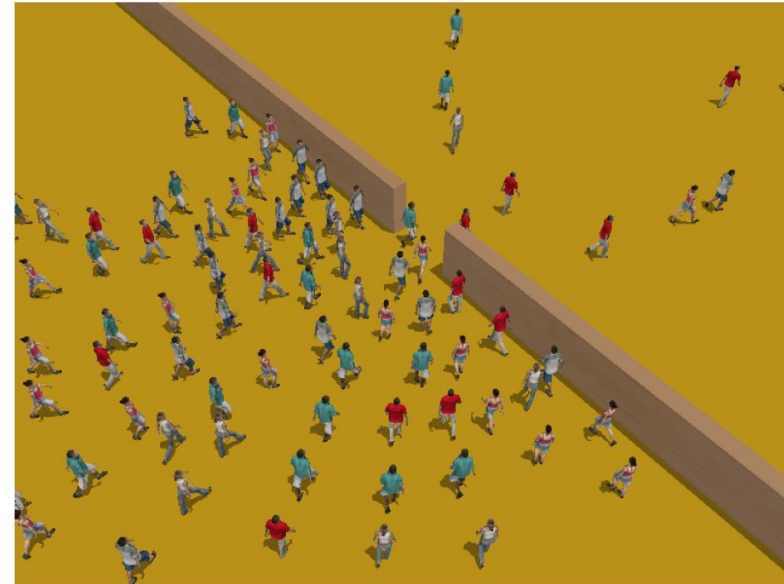
Discrete vs. Continuous

- Does the environment provide a fixed number of distinct percepts, actions, and environment states?
 - Are the values of the state variables discrete or continuous?
 - Time can also evolve in a discrete or continuous fashion.



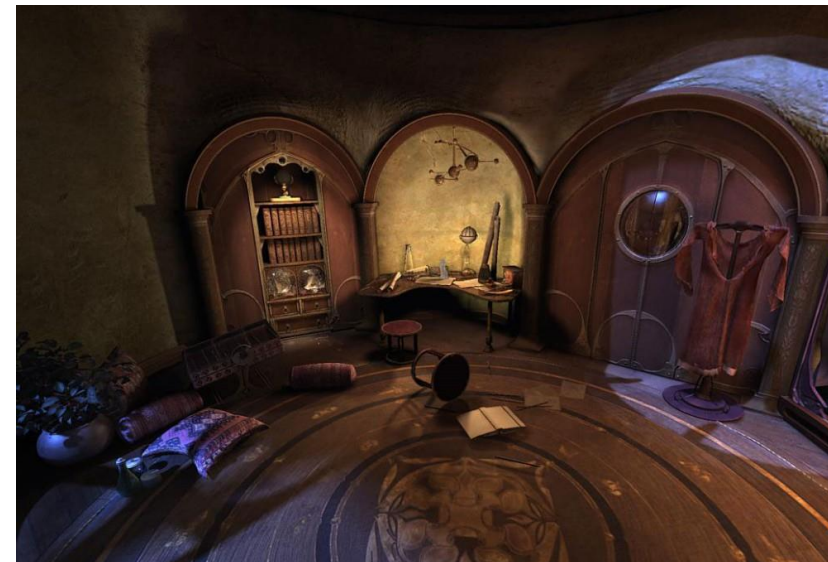
Single-agent vs. Multi-agent

- Is an agent operating by itself in the environment?



Known vs. Unknown

- Are the rules of the environment (transition model and rewards associated with states) known to the agent?
 - Strictly speaking, not a property of the environment, but of the agent's state of knowledge.



Agent Design

- The environment type largely determines the agent design:
 - Partially Observable → Agent needs internal memory to track hidden state
 - Multi-Agent → May require randomized strategies to avoid predictability
 - Stochastic → Must prepare for uncertainty and plan contingencies
 - Static → Can compute optimal decisions without time pressure
 - Continuous Time → Requires a real-time, continuously operating controller
 - Unknown Physics → Agent must explore to learn environment dynamics
 - Unknown Performance Measure → Agent must interact with a human principal to infer goals

Agent Design

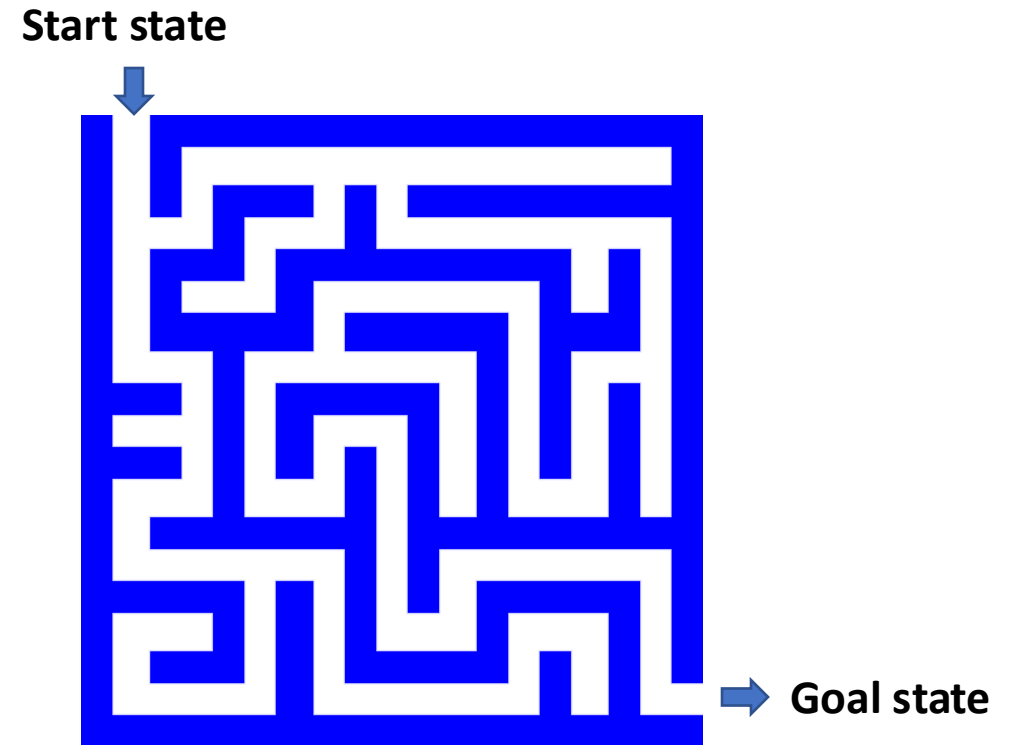
Task Environment	Observable	Agents	Deterministic	Episodic	Static	Discrete
Crossword puzzle	Fully	Single	Deterministic	Sequential	Static	Discrete
Chess with a clock	Fully	Multi	Deterministic	Sequential	Semi	Discrete
Poker	Partially	Multi	Stochastic	Sequential	Static	Discrete
Backgammon	Fully	Multi	Stochastic	Sequential	Static	Discrete
Taxi driving	Partially	Multi	Stochastic	Sequential	Dynamic	Continuous
Medical diagnosis	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Image analysis	Fully	Single	Deterministic	Episodic	Semi	Continuous
Part-picking robot	Partially	Single	Stochastic	Episodic	Dynamic	Continuous
Refinery controller	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
English tutor	Partially	Multi	Stochastic	Sequential	Dynamic	Discrete

Problem-solving Agent

- When the correct action isn't obvious, an agent may need to plan ahead to evaluate a sequence of actions that lead to a goal.
- This type of agent is called a **problem-solving agent**, and the process it uses is known as **search**.

Search Problem

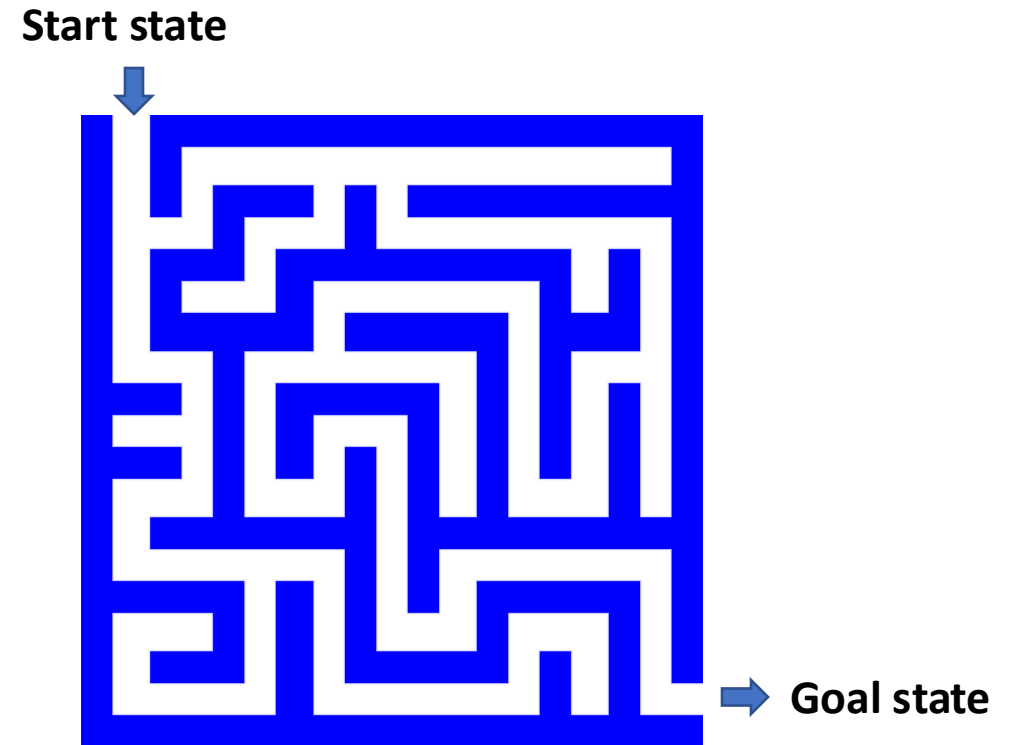
- We consider a fully observable, deterministic, discrete, and known environment.
- The agent must compute a **sequence of actions** that leads to the goal.
- Its performance is evaluated by (1) whether it reaches the goal, and (2) the cost of the path taken.



Maze: a classic example of search problem

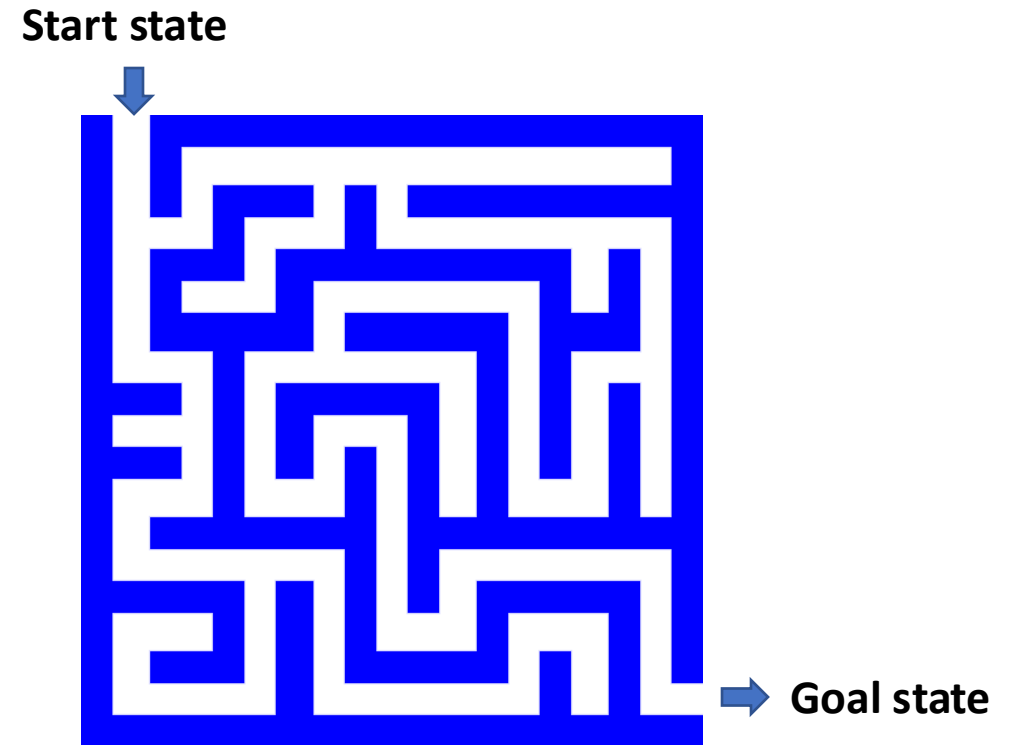
Problem Formulation

- **State space:** All possible states that the environment can be in;
- Initial state: Where the agent starts;
- Goal state: Desired target condition;
- Actions: Available choices the agent can make;
- Transition model: Defines the resulting state from applying an action in a given state;
- Action cost: Numeric cost associated with each action



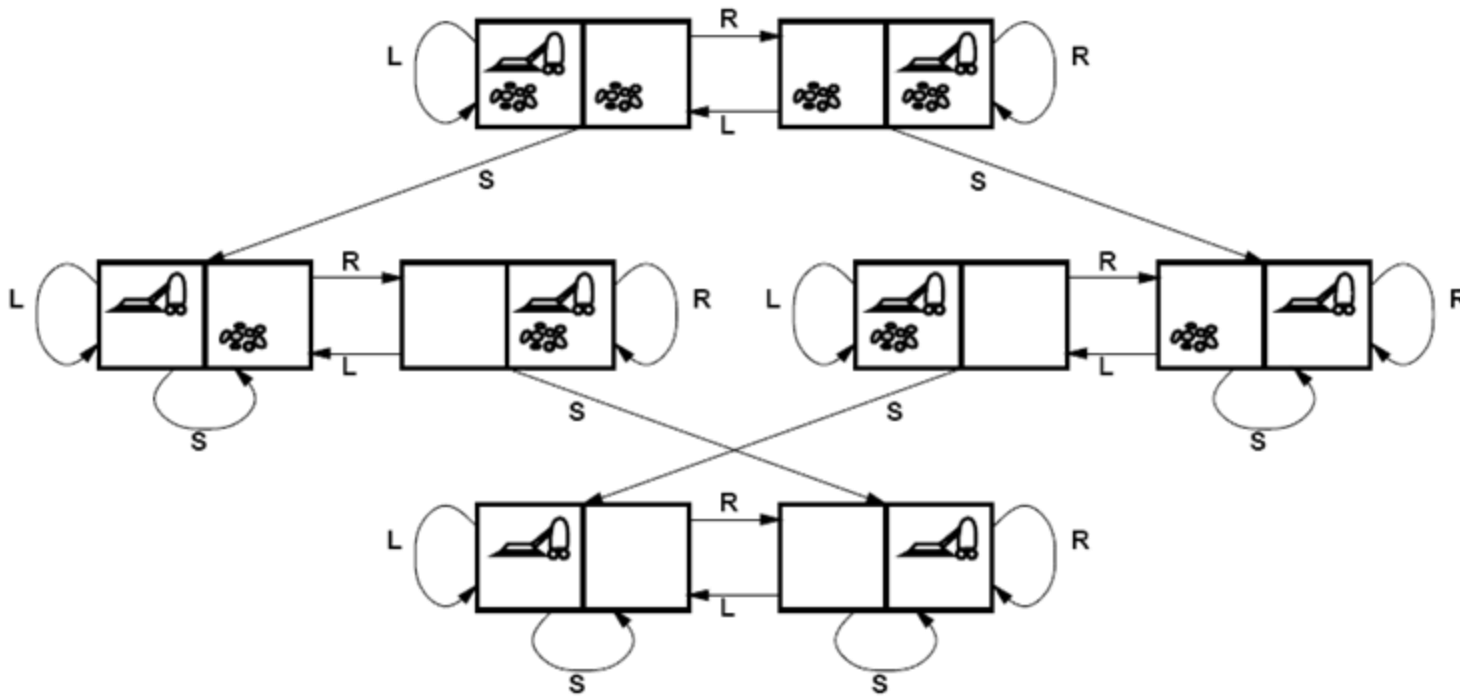
Problem Formulation

- A sequence of actions defines a **path** through the state space.
- A **solution** is a path from the initial state to a goal state.
- **Optimality**: Finds the path with the lowest total cost.
- **Completeness**: Guarantees finding a solution if one exists.
- Time / space cost, etc.



State Space

- The state space can be modeled as a **graph**, where **nodes represent states** and directed **edges represent actions** leading from one state to another.



- States
 - Agent location and dirt location;
 - How many possible states?
 - What if there are n ($n > 2$) possible locations?
- Actions
 - Left, right, clean, no-op

Example: The 8-Puzzle

- States
 - Locations of tiles
 - 8-puzzle: 181,440 states ($9!/2$)
 - 15-puzzle: ~10 trillion states
 - 24-puzzle: $\sim 10^{25}$ states
- Actions
 - Move blank left, right, up, down
- Path cost
 - 1 per move

7	2	4
5		6
8	3	1

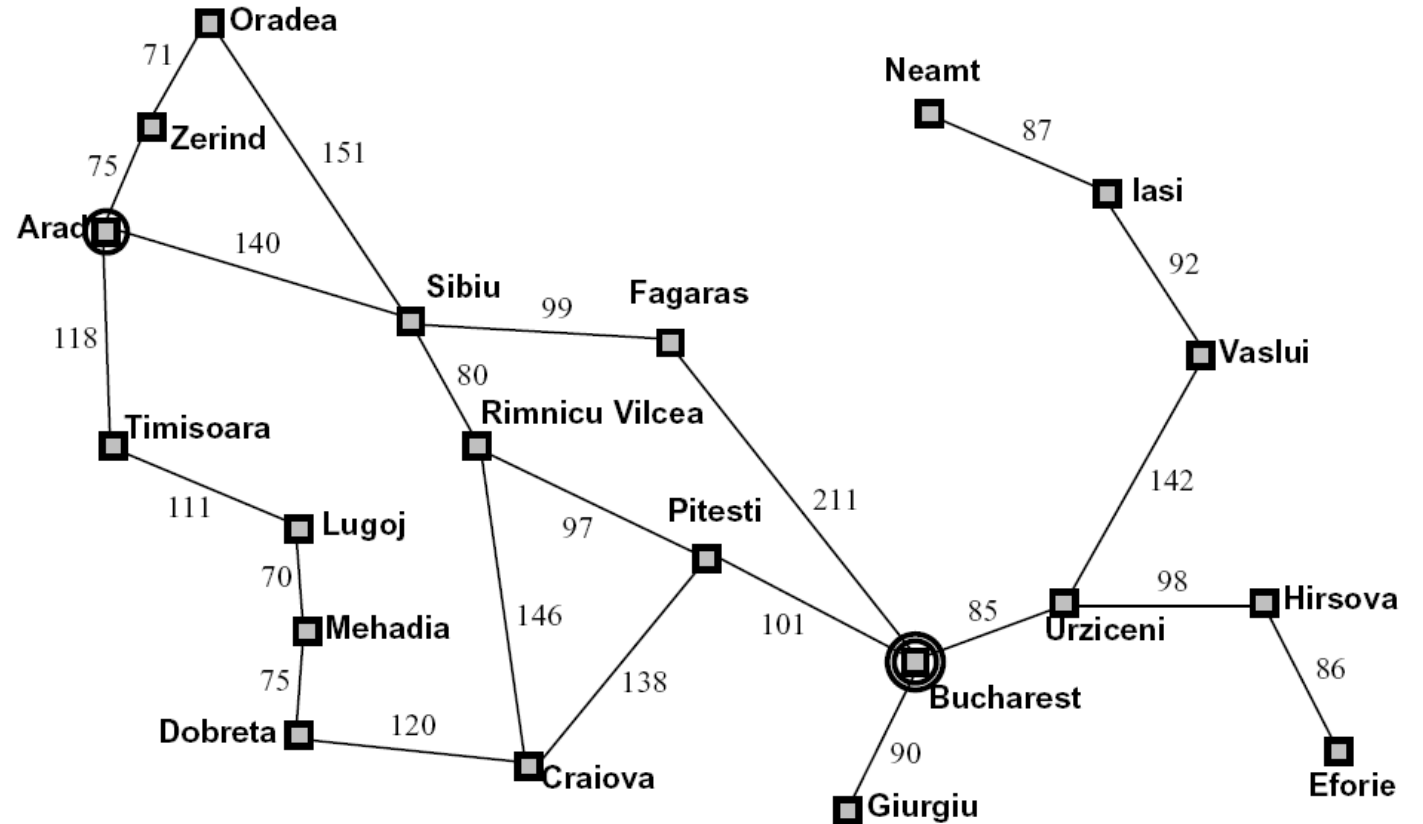
Start State

	1	2
3	4	5
6	7	8

Goal State

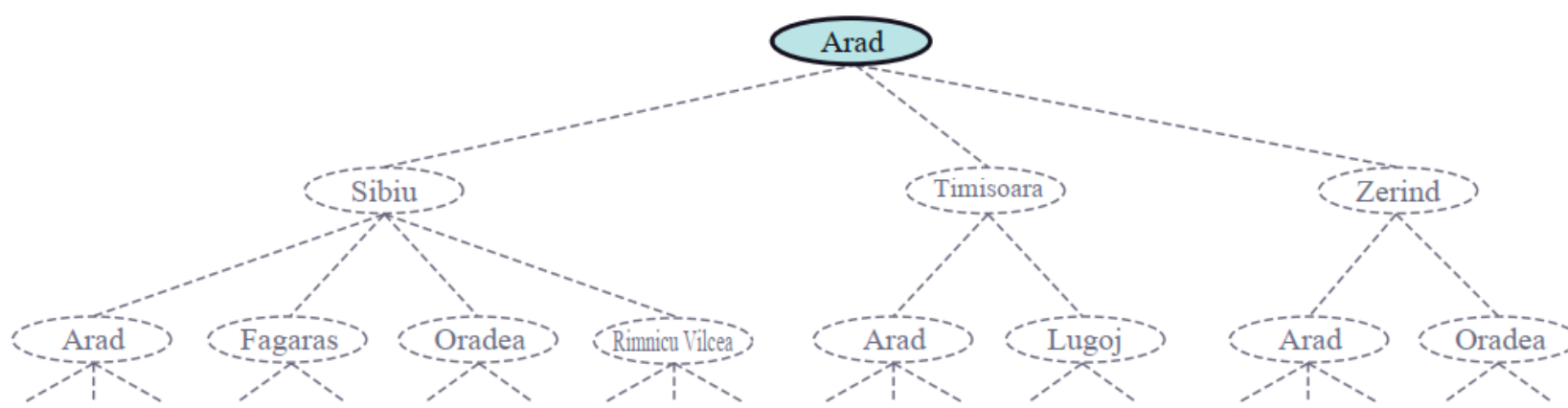
Example: Traveling in Romania

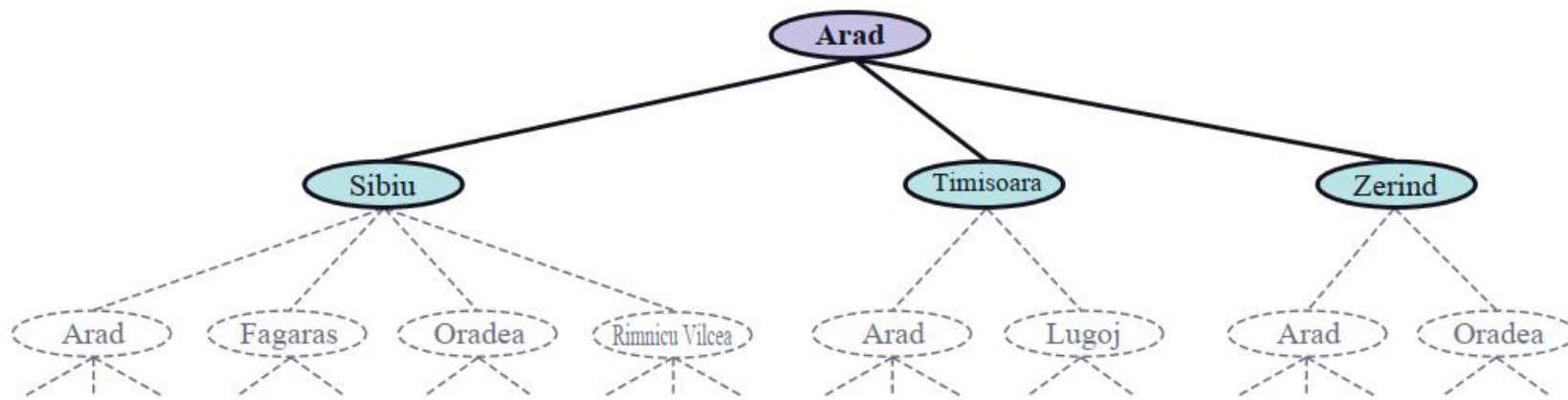
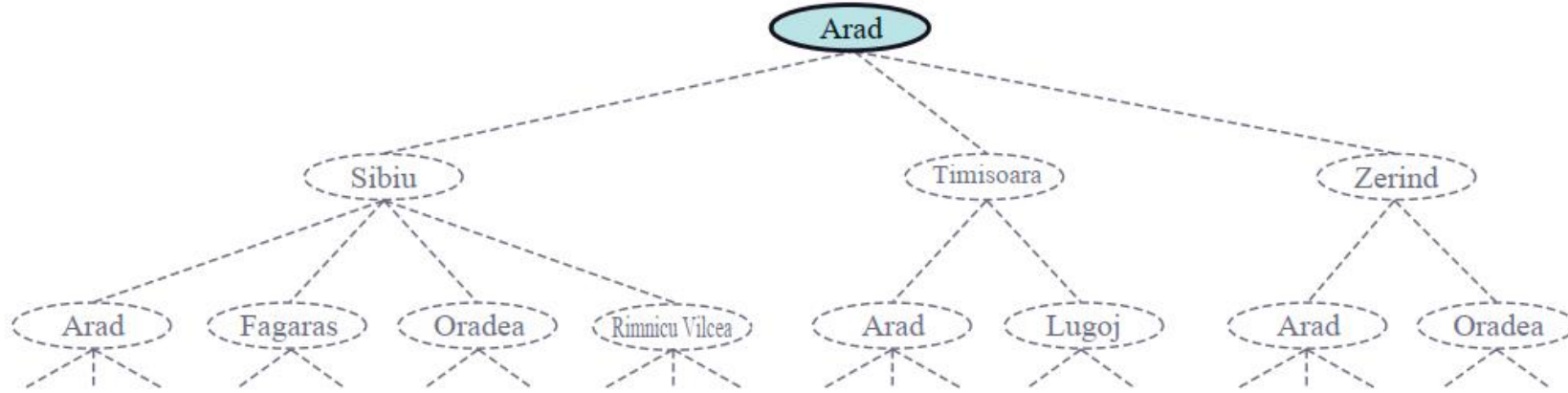
- States:
 - Cities
- Actions:
 - Roads: Go to adjacent city with cost = distance
- Start state:
 - Arad
- Goal test:
 - Is state == Bucharest?

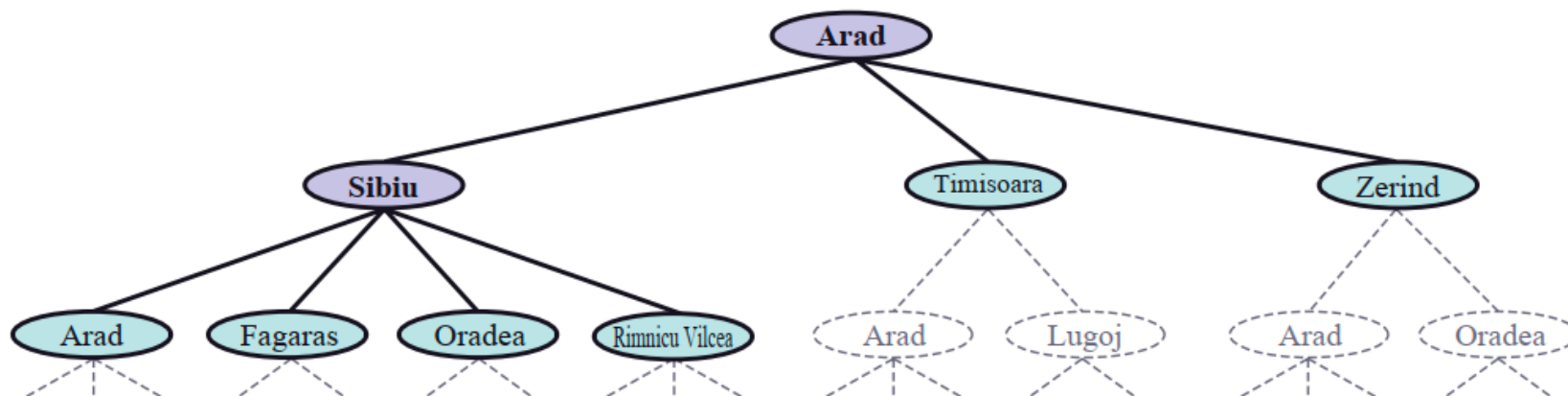
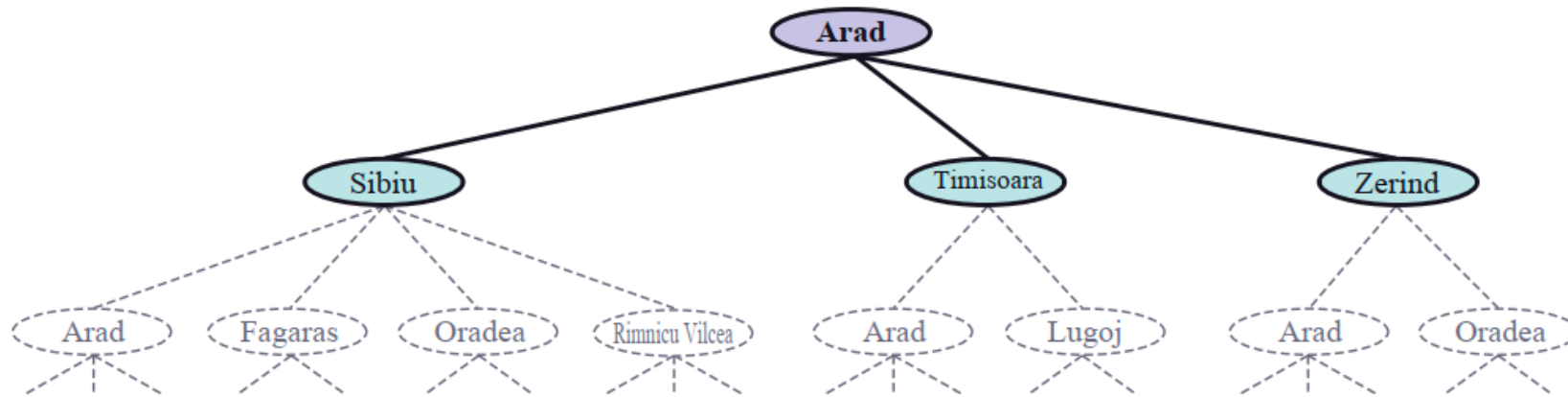
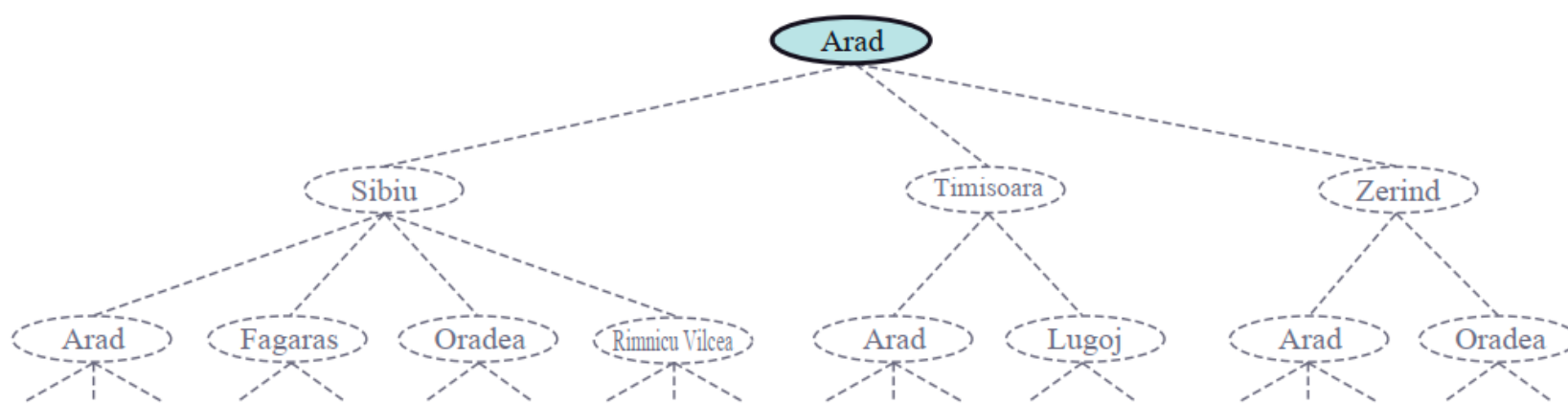


Search Tree

- We superimpose a search tree over the state space graph, forming various paths from the initial state, trying to find a path that reaches a goal state.
 - Each **node** in the search tree corresponds to a **state** in the state space.
 - The **edges** in the search tree correspond to **actions**.
 - The **root** of the tree corresponds to the **initial state** of the problem.

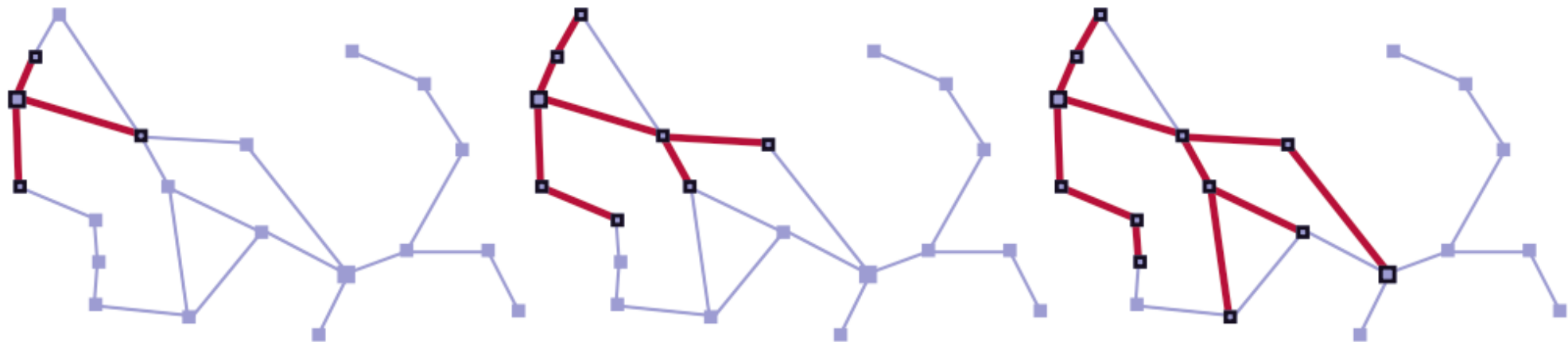






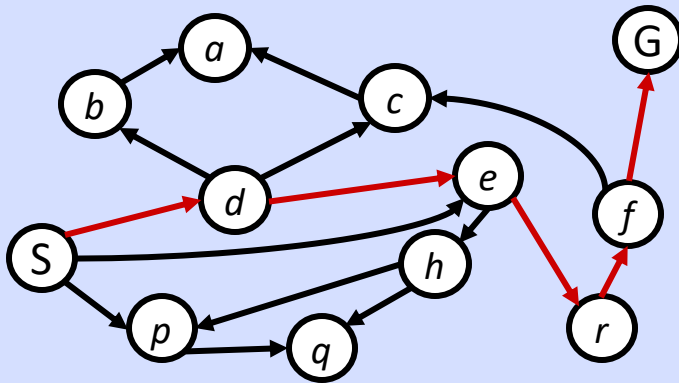
Search Tree: Algorithm

- Initialize the **frontier** using the starting state.
- While the frontier is not empty,
 - Select a node from the frontier based on the search strategy, and remove it.
 - If the node contains the goal state, return the solution.
 - Otherwise, expand the node and add its children to the frontier.



State Space Graph vs. Search Tree

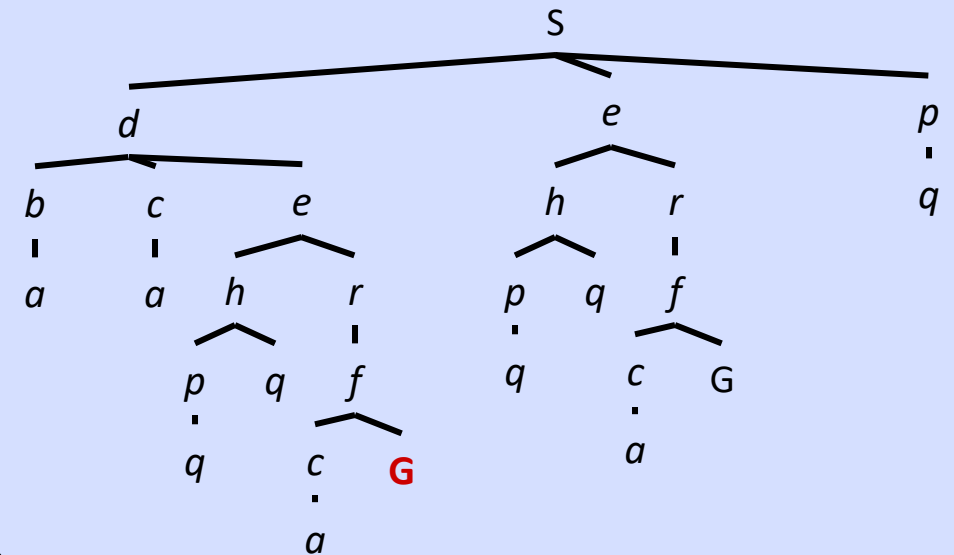
State Space Graph



Each (leaf) node in the search tree corresponds to an entire PATH in the state space graph.

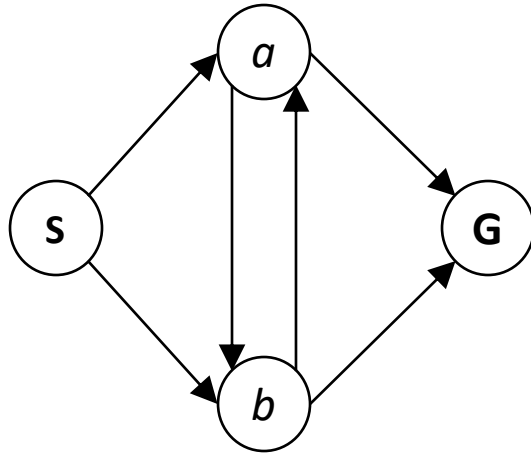
We construct only what we need on demand

Search Tree

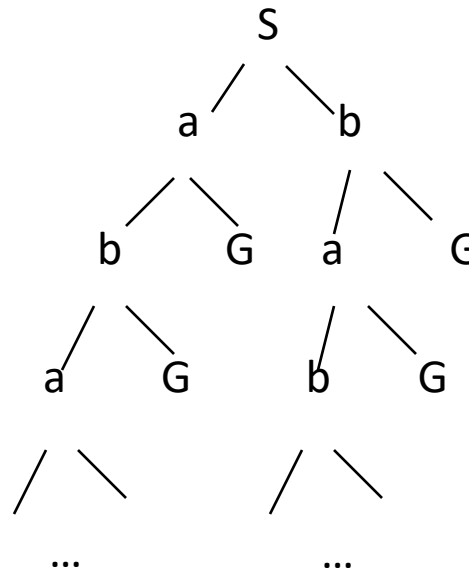


State Space Graph vs. Search Tree

Consider this 4-state graph:



How big is its search tree (from S)?



Important: Lots of repeated structure in the search tree!

For most problems, we can never actually build the whole tree.

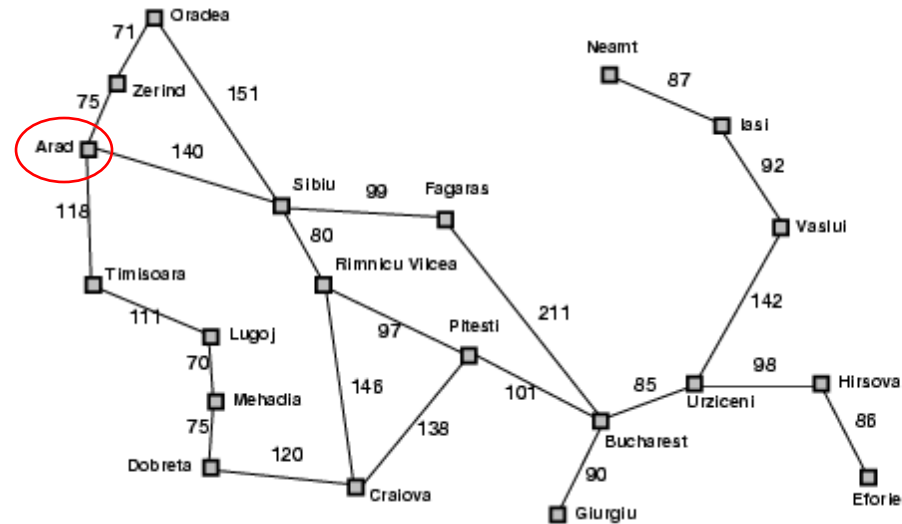
Search Tree: Algorithm Revisited

- Initialize the **frontier** using the starting state.
- While the frontier is not empty,
 - Select a node from the frontier based on the search strategy, and remove it.
 - If the node contains the goal state, return the solution.
 - Otherwise, expand the node and add its children to the frontier.
- To handle repeated states:
 - When expanding a node, add its state to the explored set and avoid re-adding explored states to the frontier.
 - When adding a node to the frontier, check if the same state already exists with a higher path cost; if so, replace it with the lower-cost version.

Search without Repeated States



Start: Arad
Goal: Bucharest

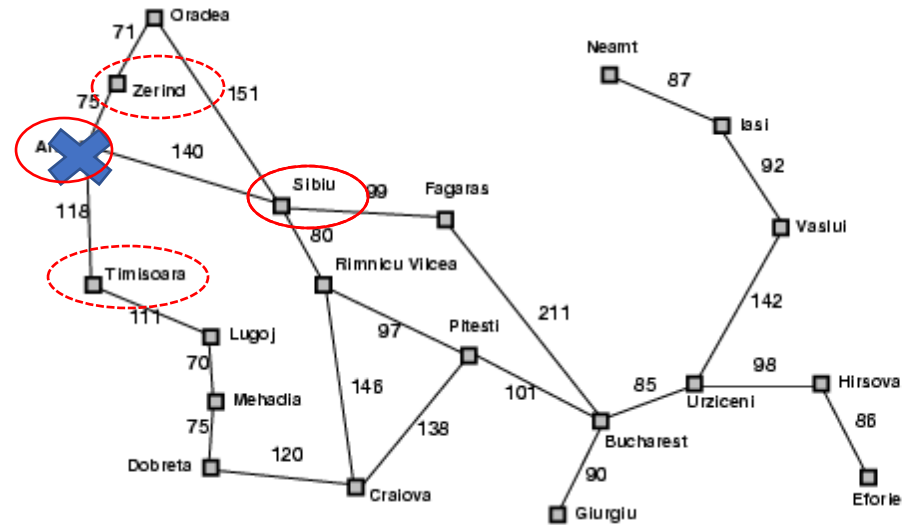


Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Search without Repeated States



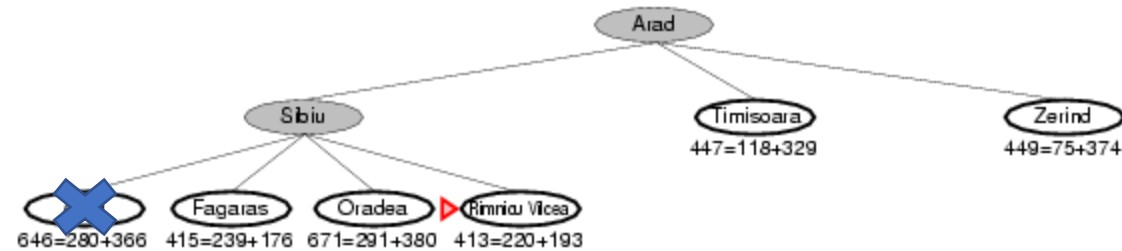
Start: Arad
Goal: Bucharest



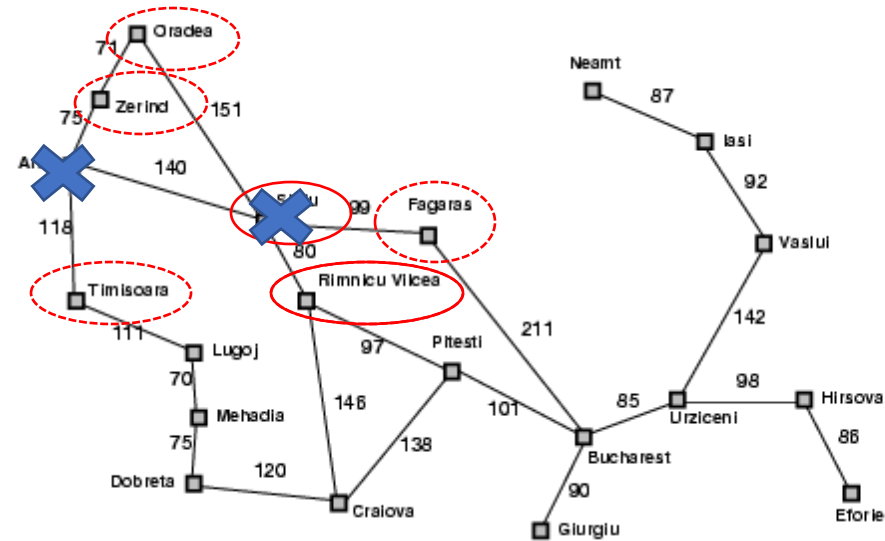
Straight-line distance
to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Search without Repeated States



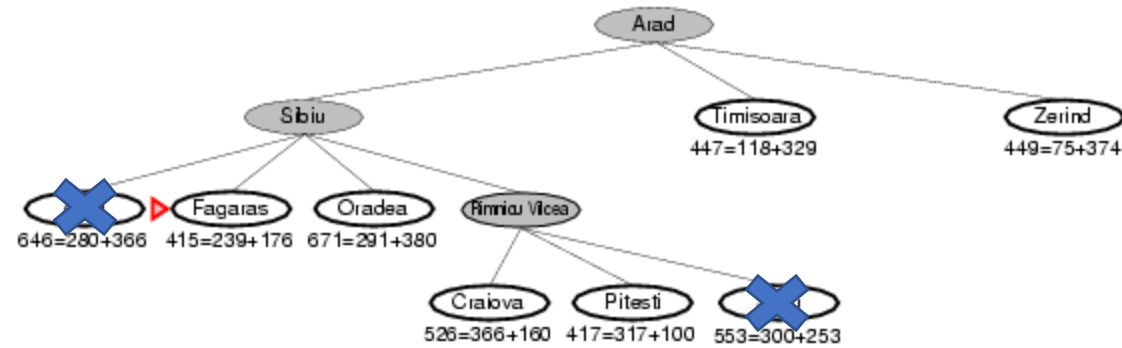
Start: Arad
Goal: Bucharest



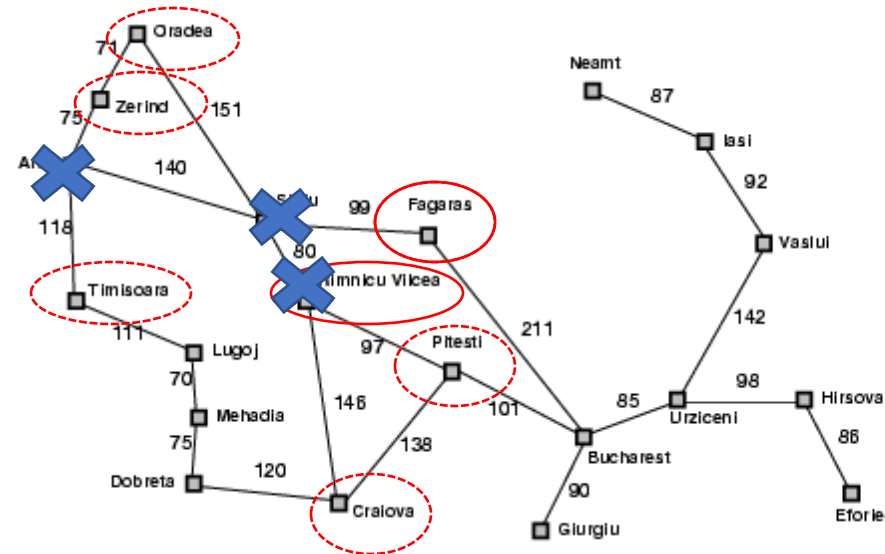
Straight-line distance
to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Search without Repeated States



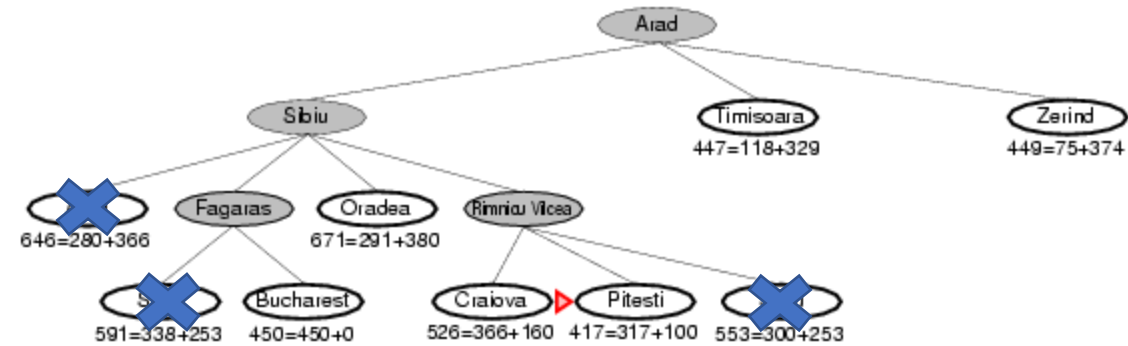
Start: Arad
Goal: Bucharest



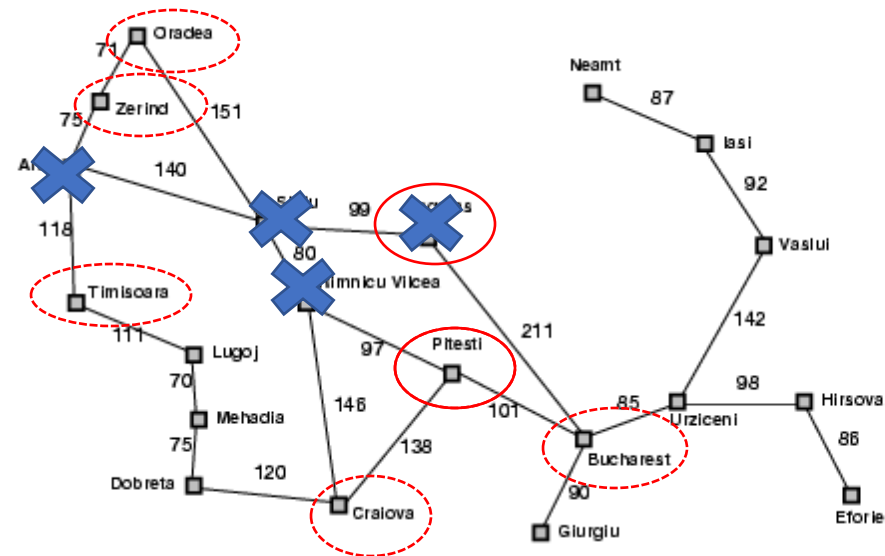
Straight-line distance
to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Search without Repeated States



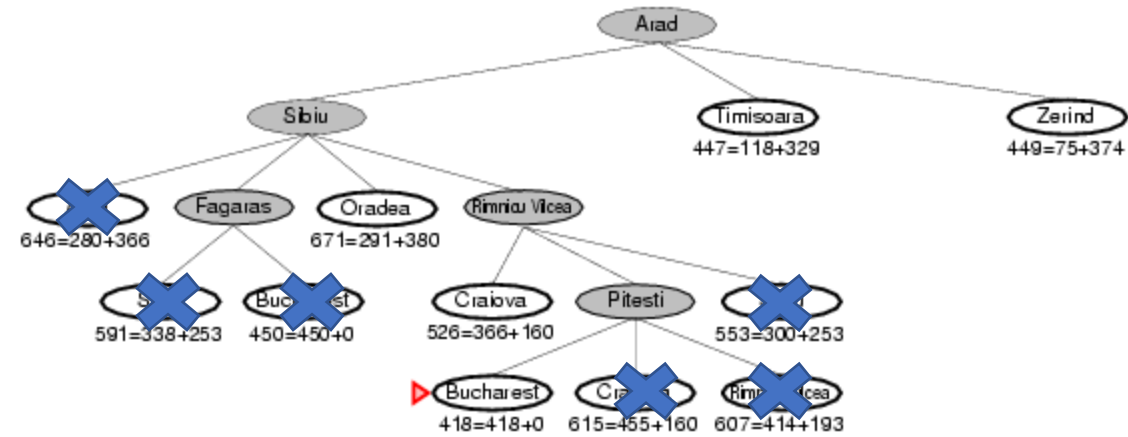
Start: Arad
Goal: Bucharest



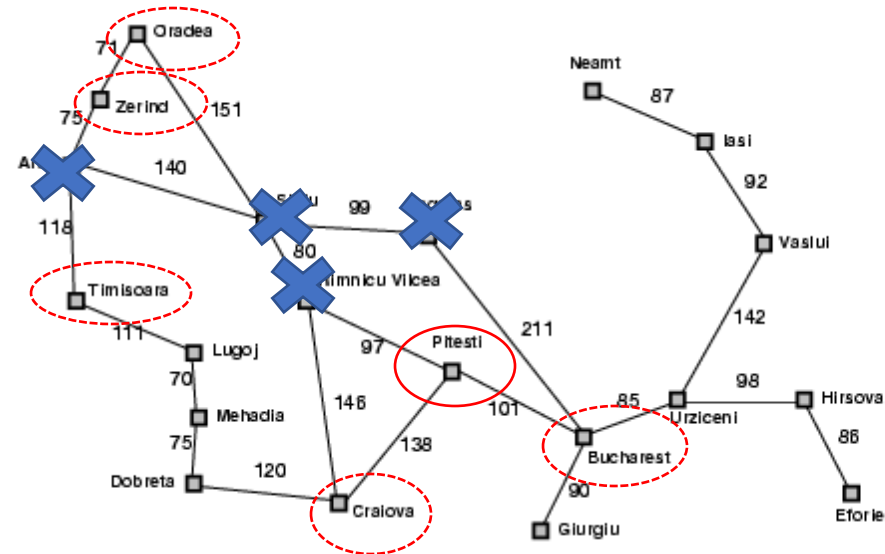
Straight-line distance
to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Search without Repeated States



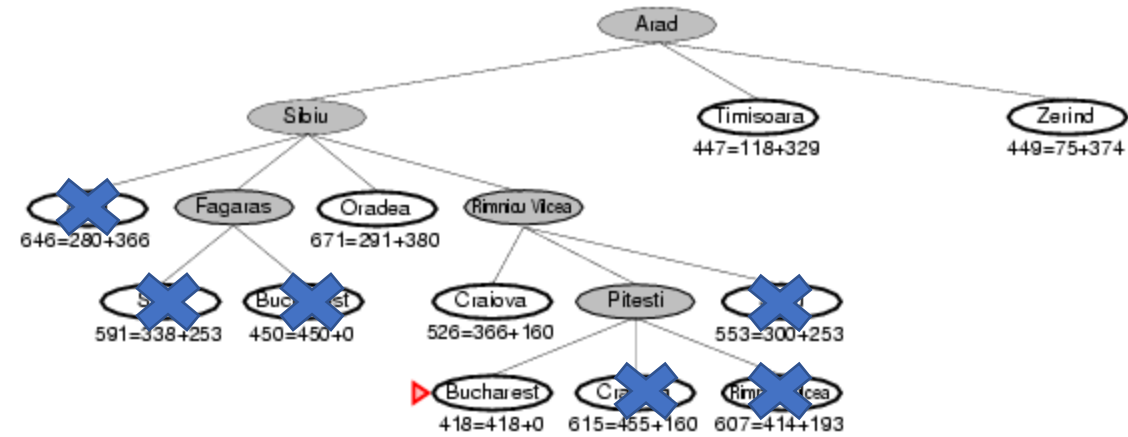
Start: Arad
Goal: Bucharest



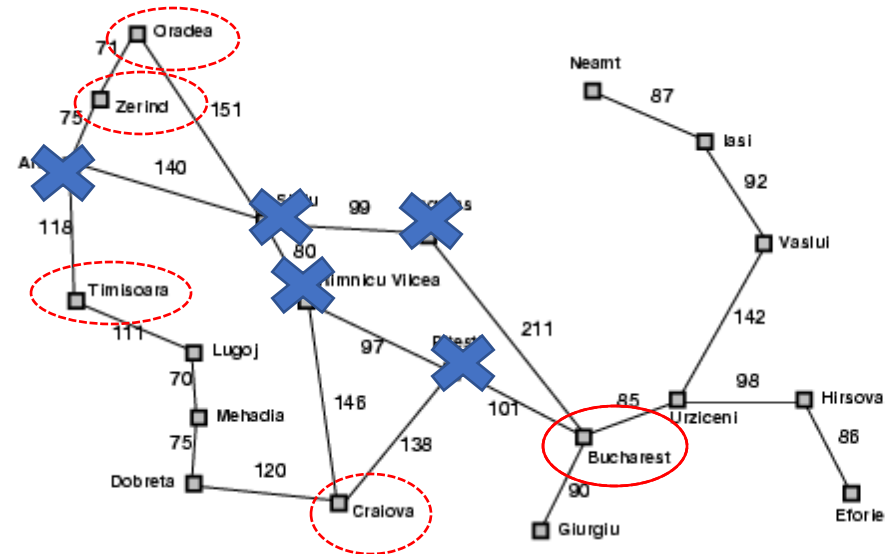
Straight-line distance
to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Search without Repeated States



Start: Arad
Goal: Bucharest



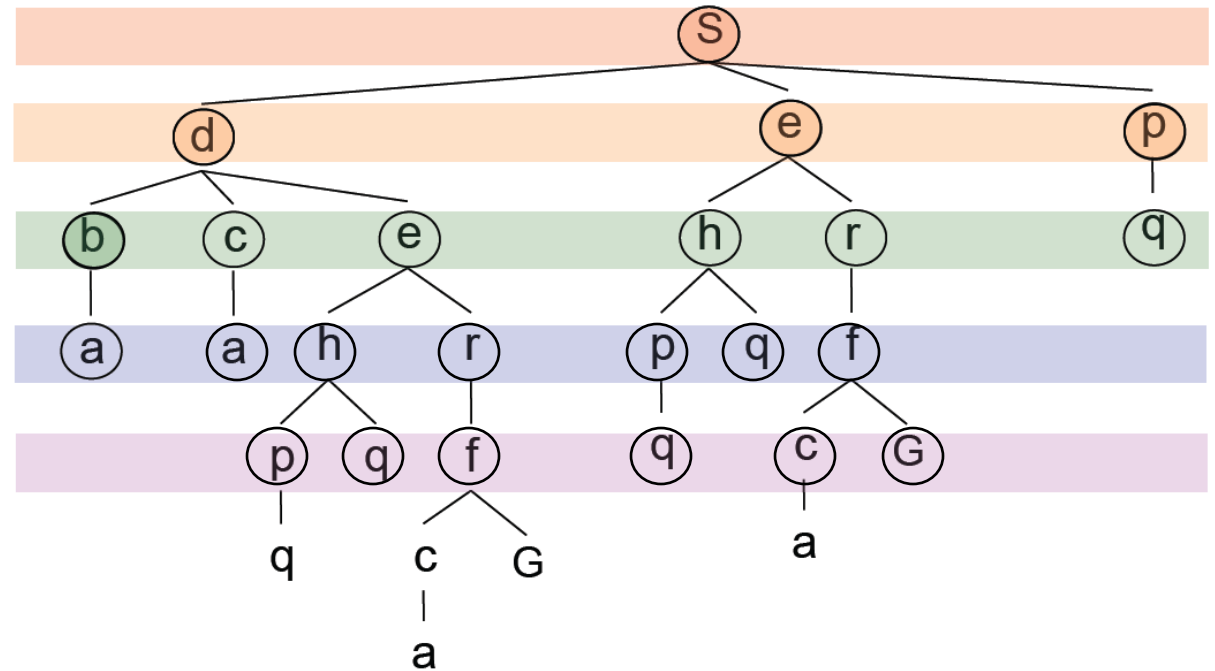
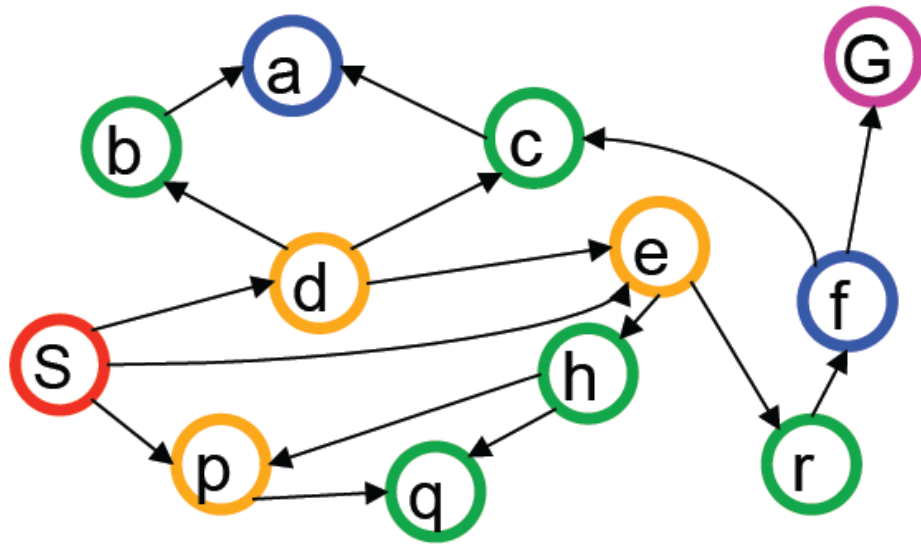
Straight-line distance
to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Search Strategy

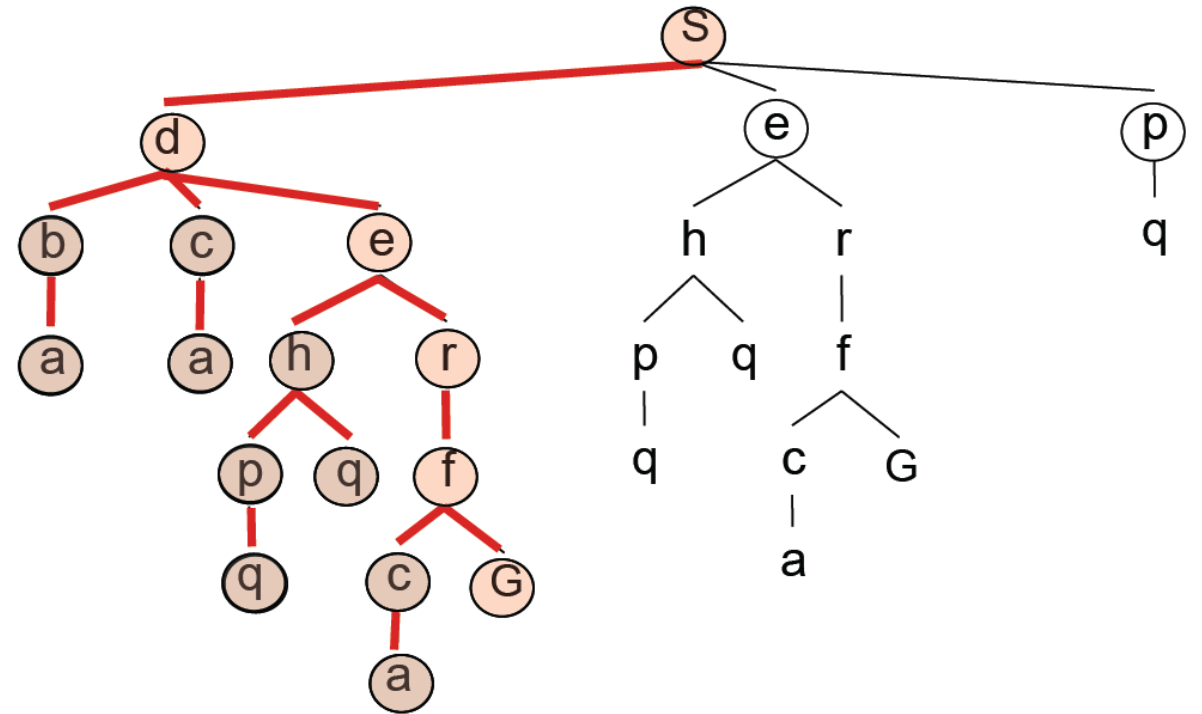
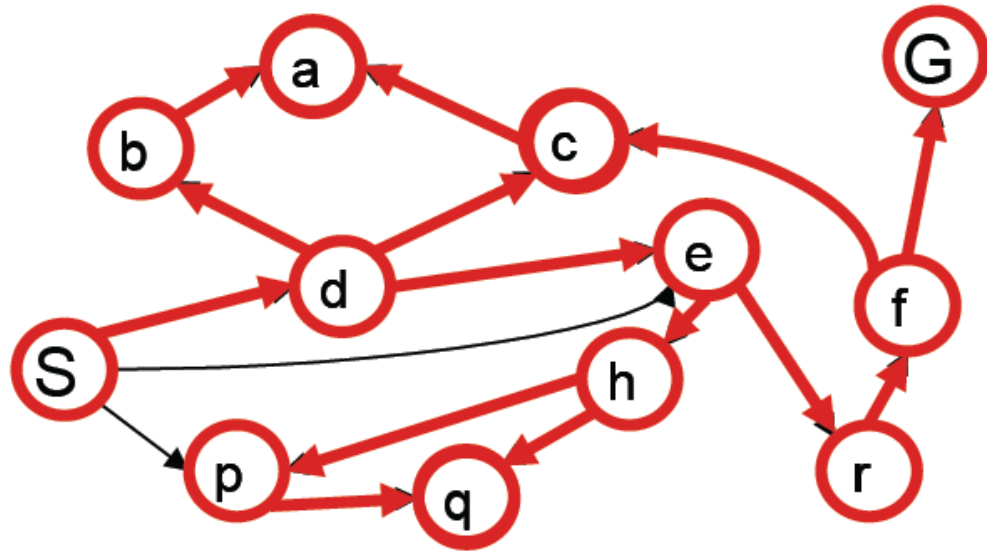
- **Search strategy** determines **the order of node expansion**, which is critical to the efficiency and outcome of the search.
- **Uninformed** search: Uses only the information provided in the problem definition.
 - Breadth-First Search (BFS), Depth-First Search (DFS), Iterative Deepening Search (IDS), Uniform-Cost Search (UCS)
- **Informed** search: Uses additional knowledge or **heuristics** to guide the search more efficiently toward the goal.
 - Greedy Best-First Search, A* Search

Breadth-First Search (BFS)



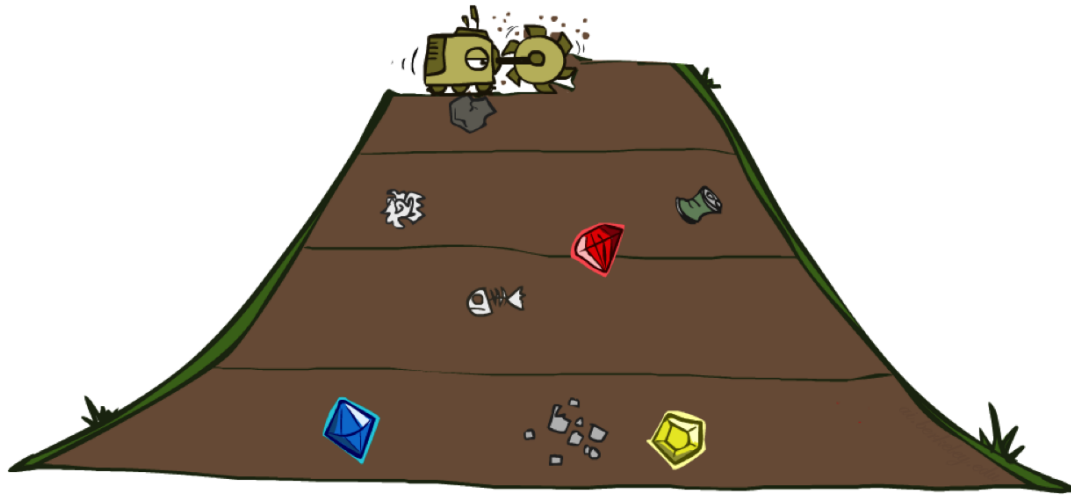
Expansion order: (S,d,e,p,b,c,e,h,r,q,a,a, h,r,p,q,f,p,q,f,q,c,G)

Depth-First Search (DFS)



Expansion order: (d,b,a,c,a,e,h,p,q,q, r,f,c,a,G)

Breadth-First Search (BFS)



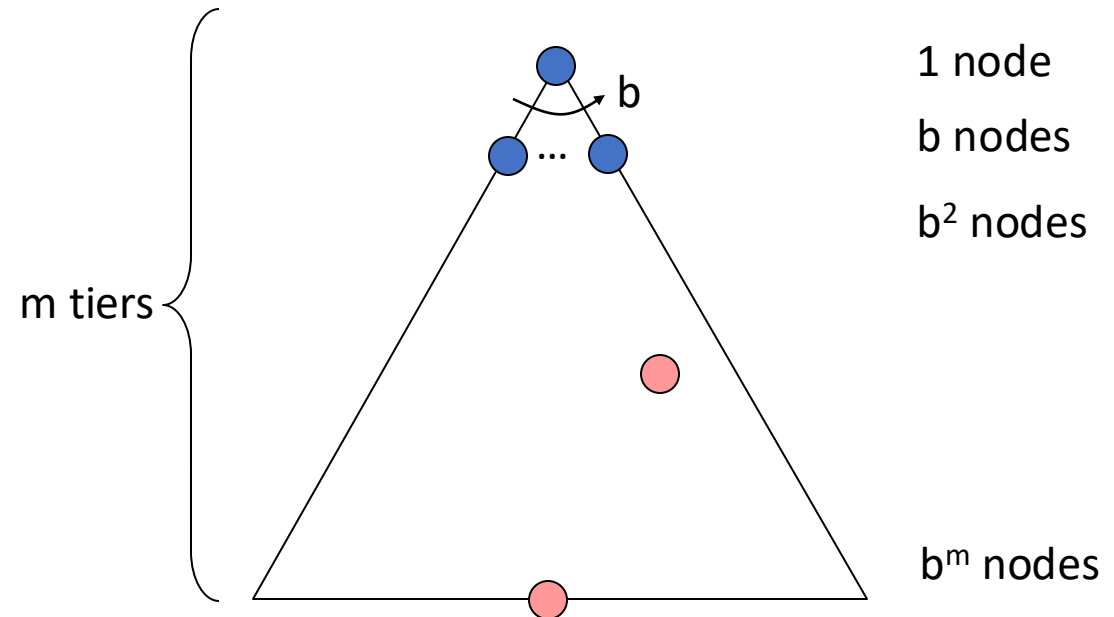
Depth-First Search (DFS)



Performance Measure of Search Strategies

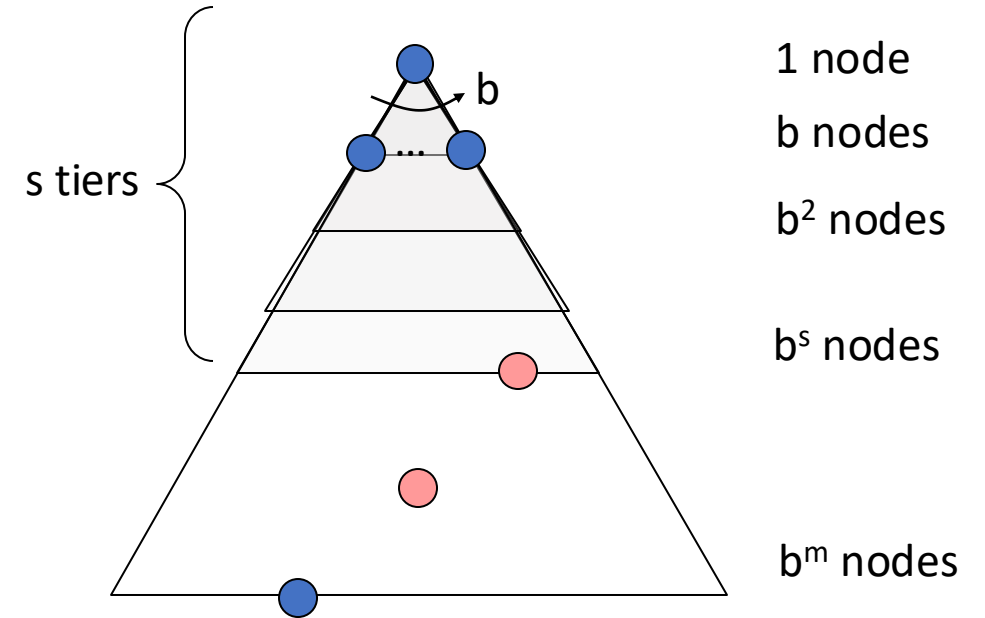
- **Complete**: Will find a solution if one exists.
- **Optimal**: Will find the lowest-cost solution among all possible paths.
- Search tree overview

- **b** is the branching factor
- **m** is the maximum depth
- Solutions may exist at different depths in the tree.
- Total number of nodes in the tree
 $1 + b + b^2 + \dots + b^m = O(b^m)$



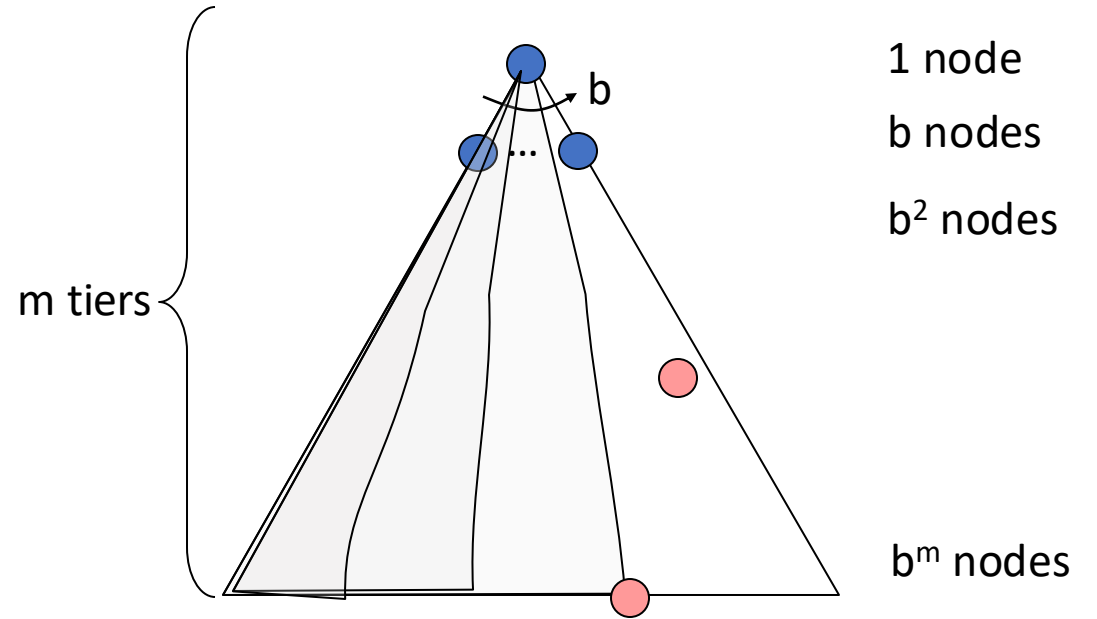
Performance of BFS

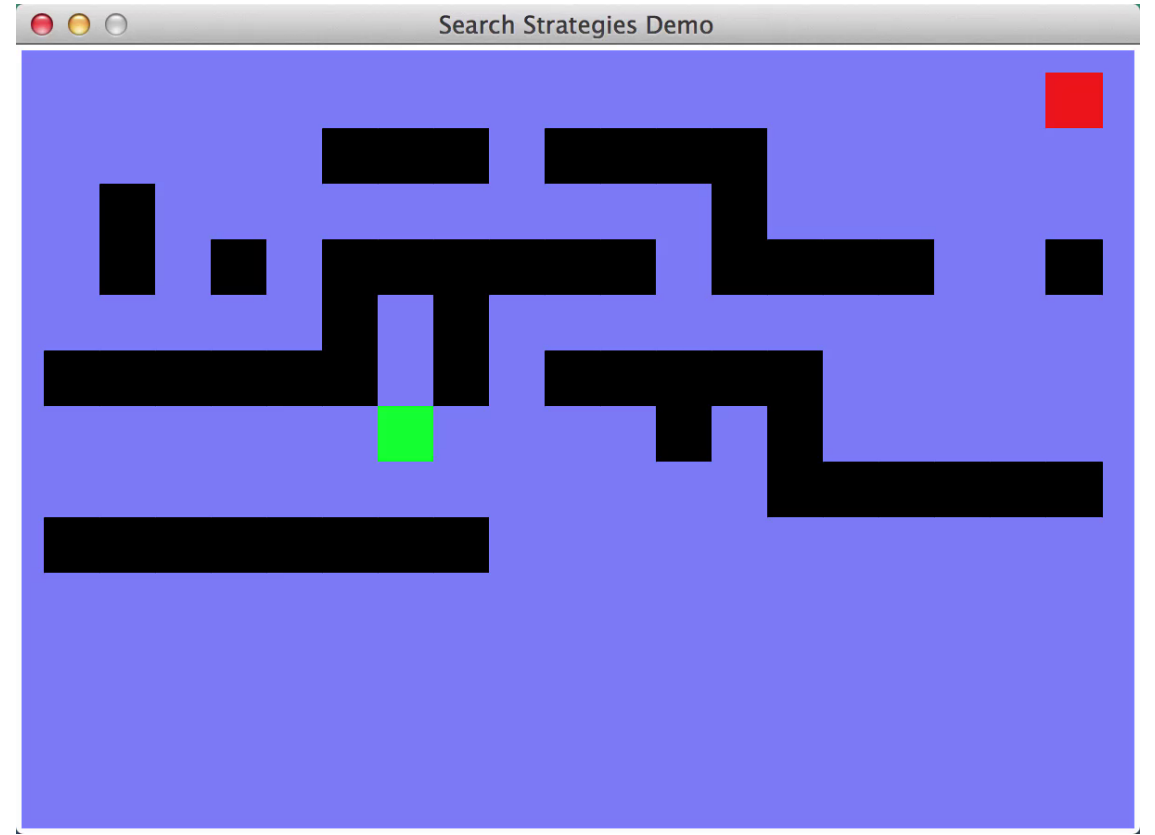
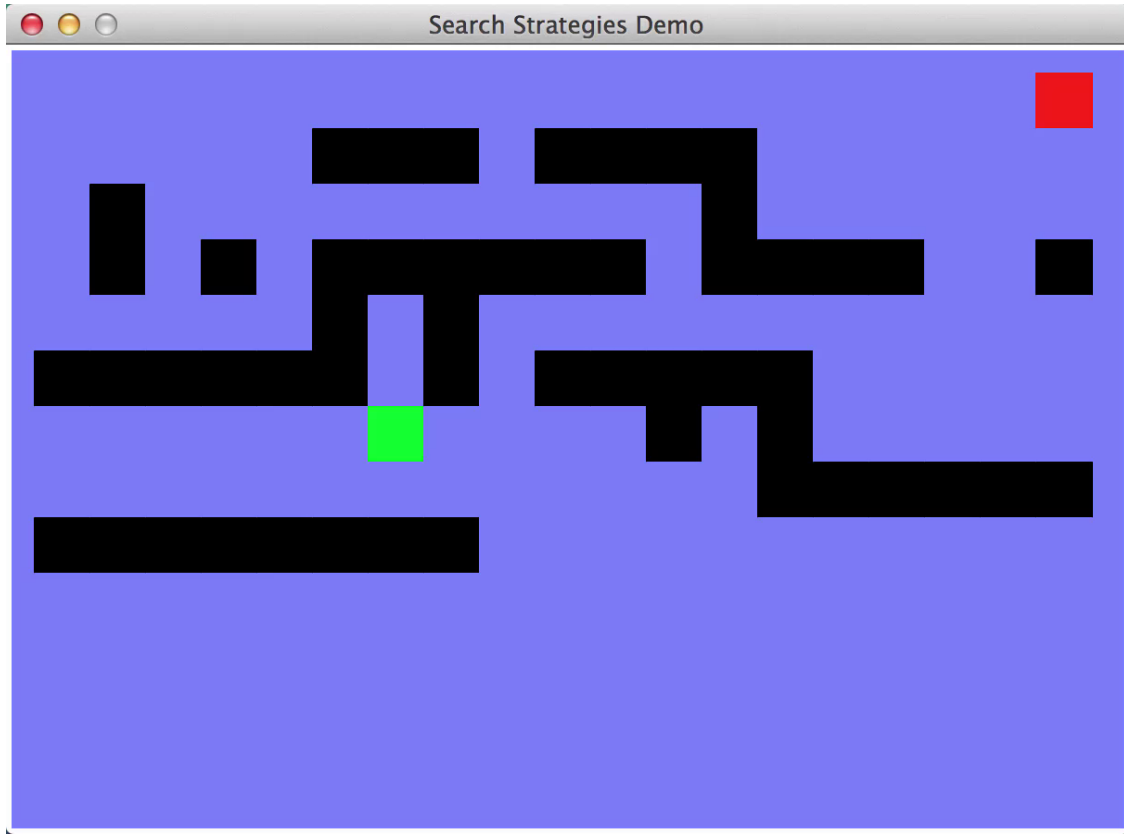
- Completeness:
 - Yes: if a solution exists, BFS will find it (since s is finite).
- Optimality:
 - Yes: if all actions have equal cost, BFS guarantees the shortest path.
- Time and space complexity:
 - $O(b^s)$



Performance of DFS

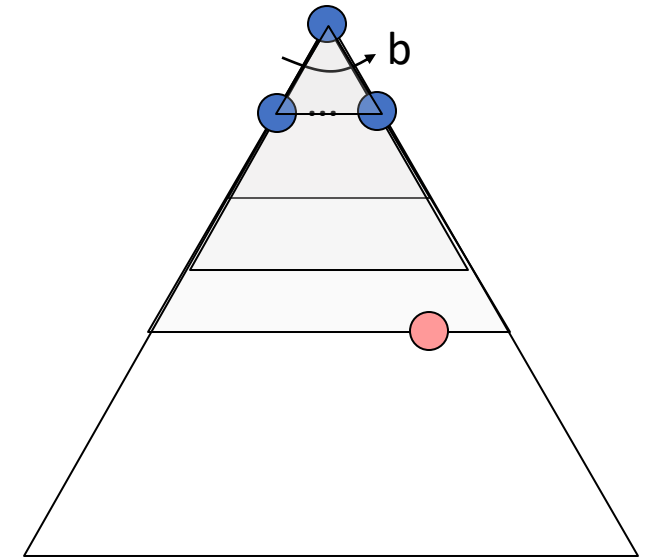
- Completeness:
 - Not guaranteed: if the search goes down an infinite path (e.g., in a cycle), it may never find a solution.
- Optimality:
 - No: DFS does not guarantee the lowest-cost or shortest solution.
- Time complexity:
 - $O(b^m)$ in the worst case
- Space complexity:
 - $O(bm)$





Iterative Deepening Search (IDS)

- Idea: get DFS's space efficiency with BFS's shallow-solution advantage
 - Run depth-limited DFS with limit = 1
 - If no solution is found, increase the limit and run again:
 - Limit = 2
 - Limit = 3
 - ...
 - Continue until a solution is found



- Space efficiency of DFS
- Completeness and optimality (like BFS, when costs are uniform)

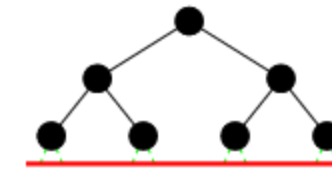
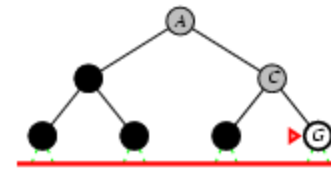
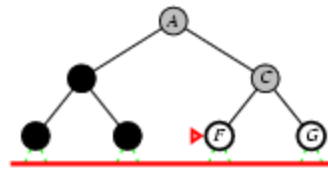
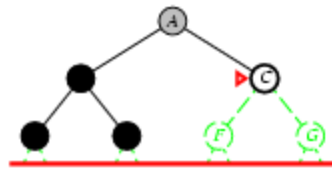
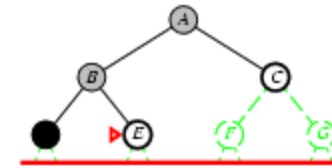
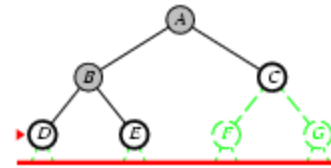
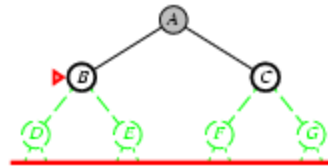
Limit = 0



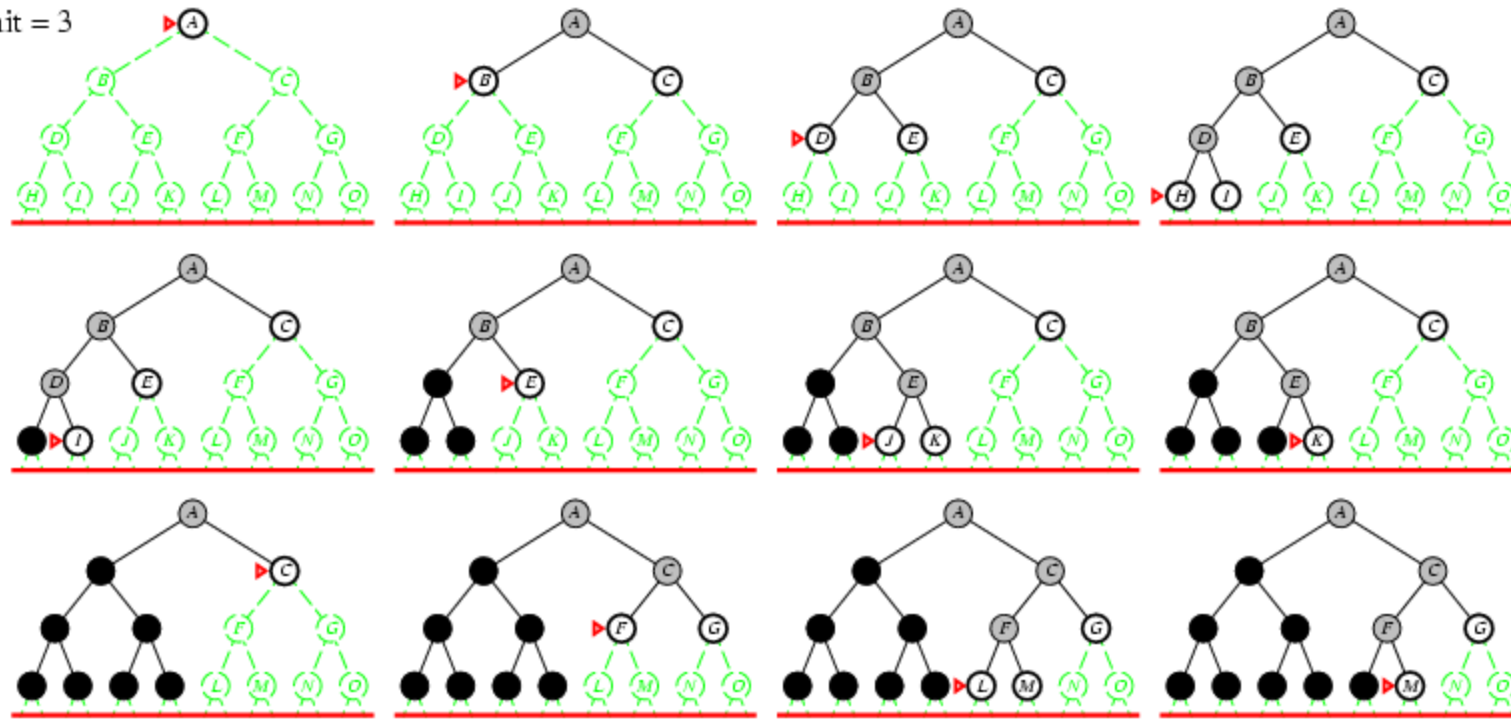
Limit = 1



Limit = 2

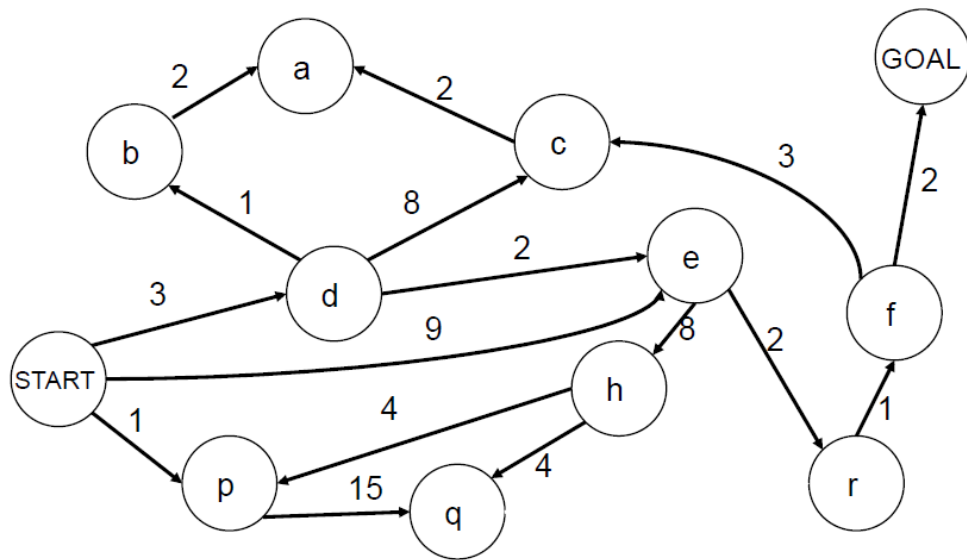


Limit = 3

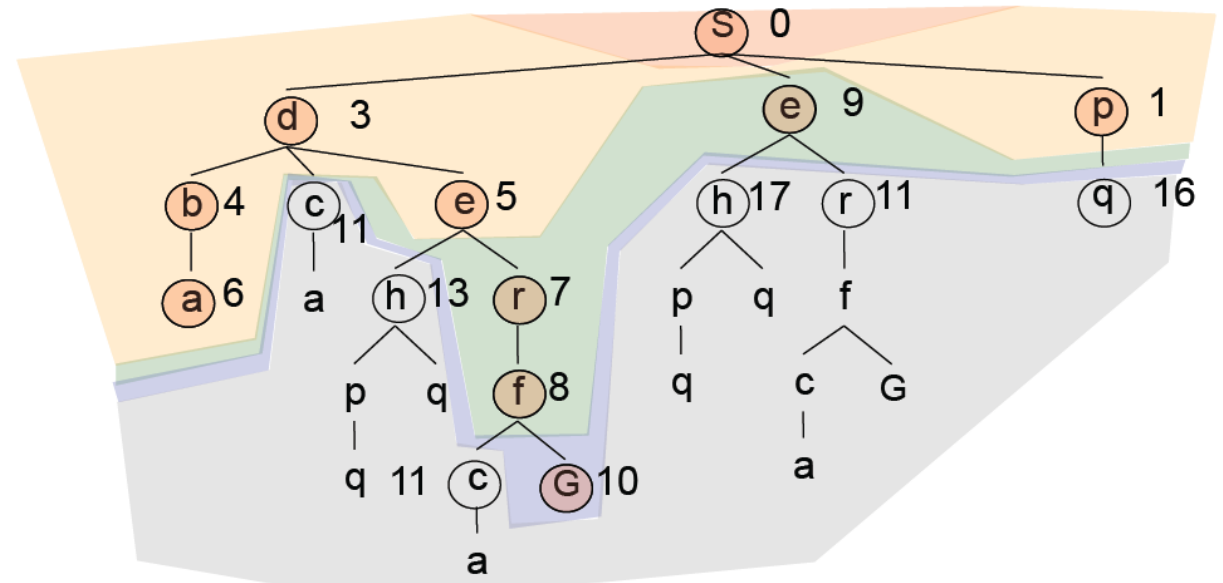


Uniform-Cost Search (UCS)

- For each node in the frontier:
 - Store the total path cost from the initial state to that node;
 - Always expand the node with the lowest path cost.



The frontier is implemented as a priority queue, ordered by path cost.



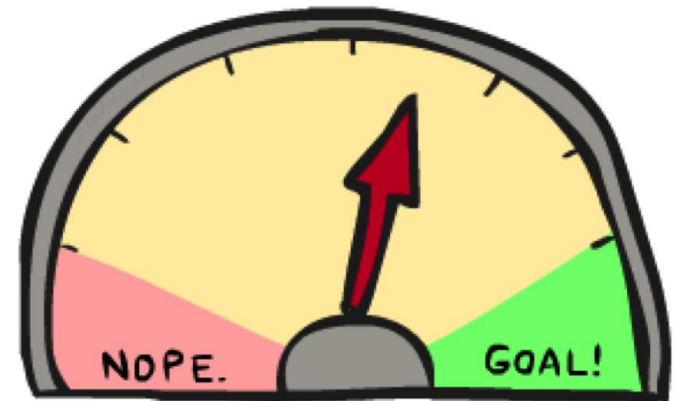
Expansion order: (S,p,d,b,e,a,r,f,e,G)

Uninformed Search vs. Informed Search

- **Uninformed** search: Uses only the information provided in the problem definition.
 - Breadth-First Search (BFS), Depth-First Search (DFS), Iterative Deepening Search (IDS), Uniform-Cost Search (UCS)
- **Informed** search: Unlike uninformed search, which relies only on the problem definition, informed search uses additional knowledge to guide exploration.

Guiding Search with Heuristics

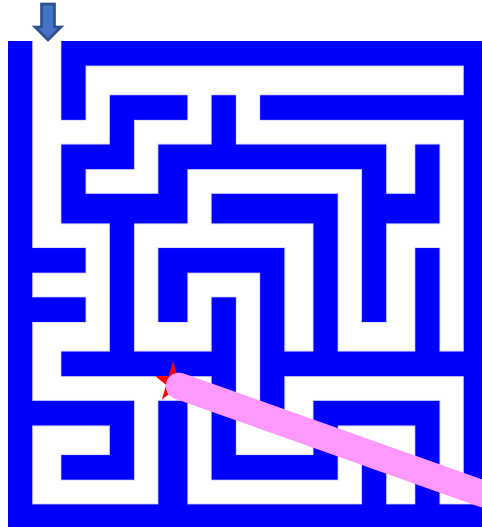
- The key idea is to give the algorithm “hints” about which states are more promising.
 - A **heuristic function** estimates how close a given state is to the goal.
 - The search algorithm uses this estimate to rank nodes and expand the most promising one first.



Greedy Best-first Search

- Always expand the node with the lowest heuristic value $h(n)$
 - The heuristic function estimates how close a state is to the goal.
 - This strategy focuses on the most promising-looking node, without considering the actual cost from the start state.

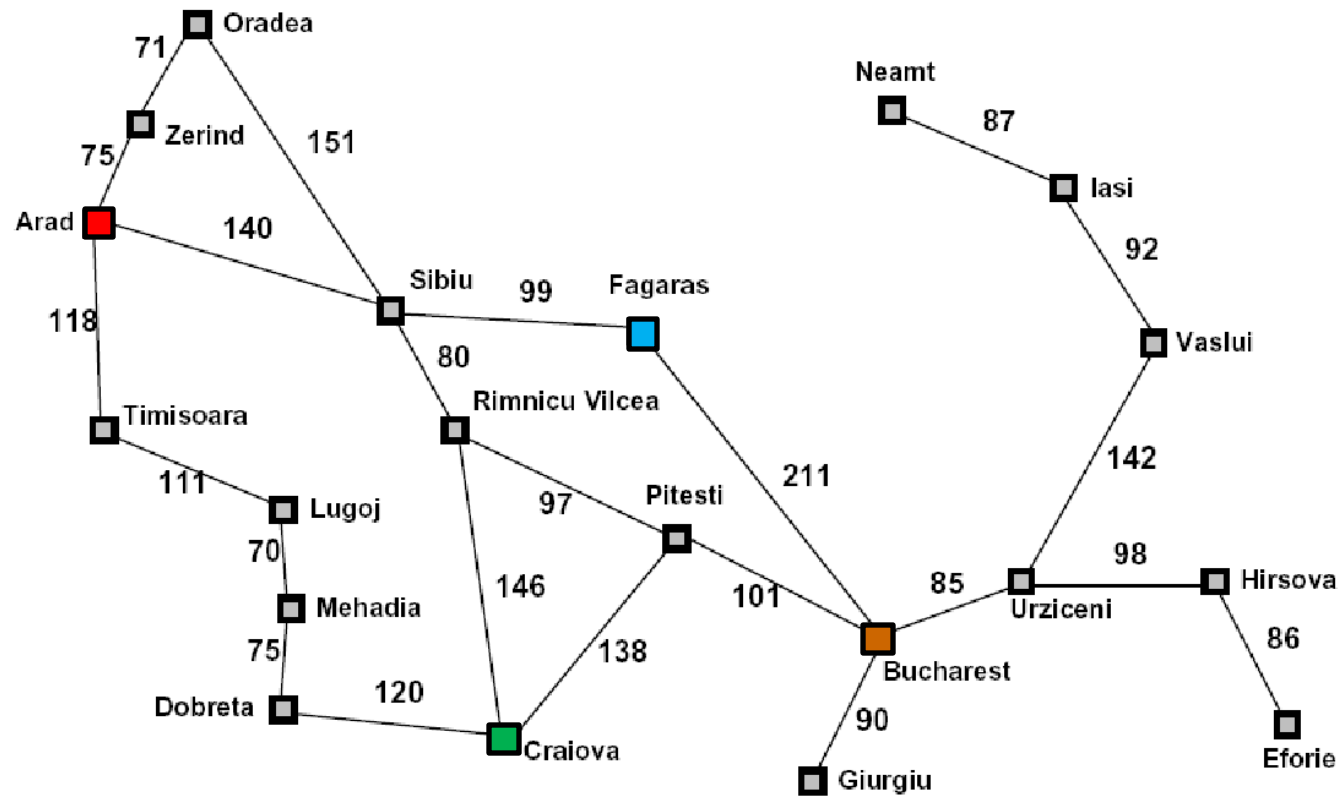
Start state



We can use the straight-line distance from the current state to the goal state as the heuristic function.

Goal state

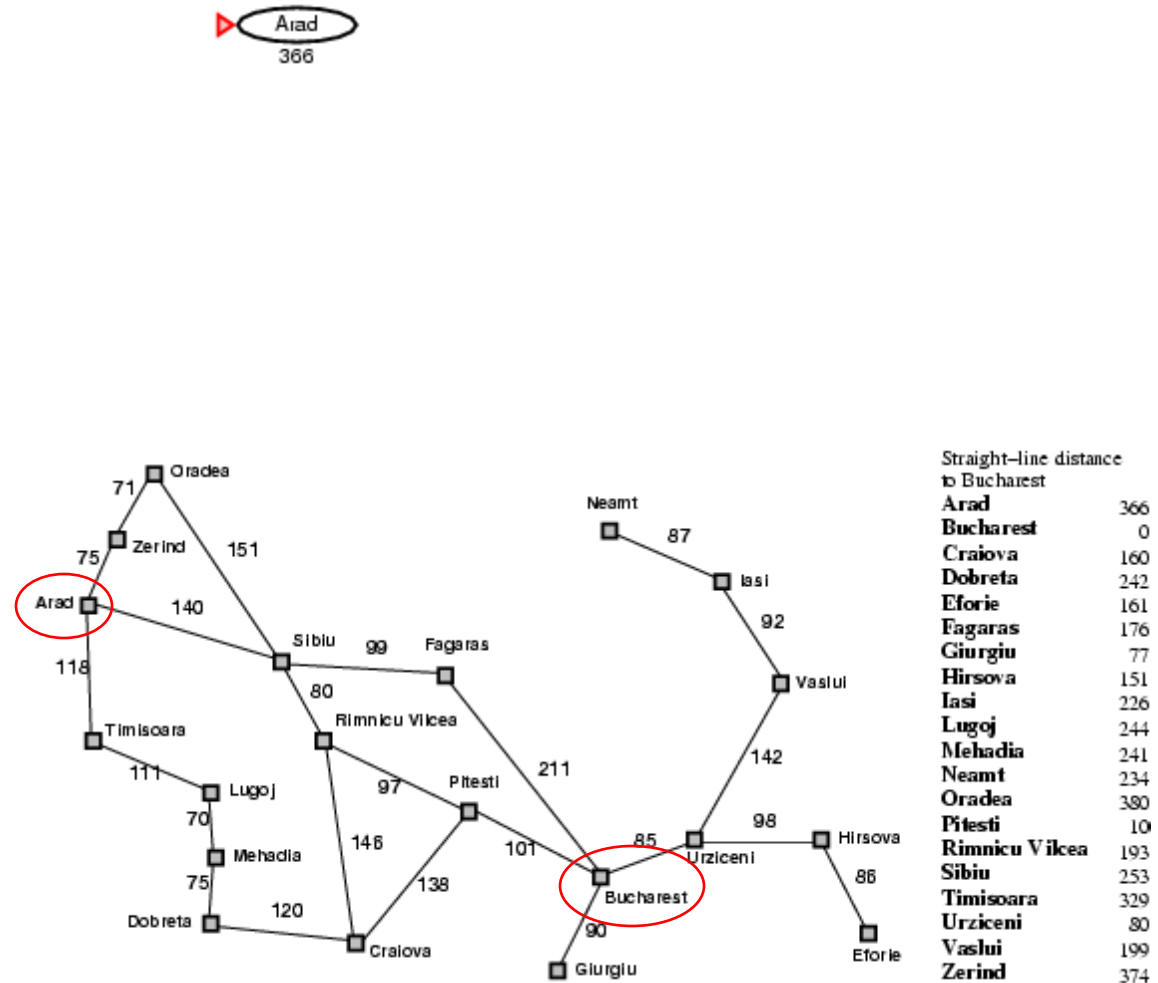
Greedy Best-first Search: Example



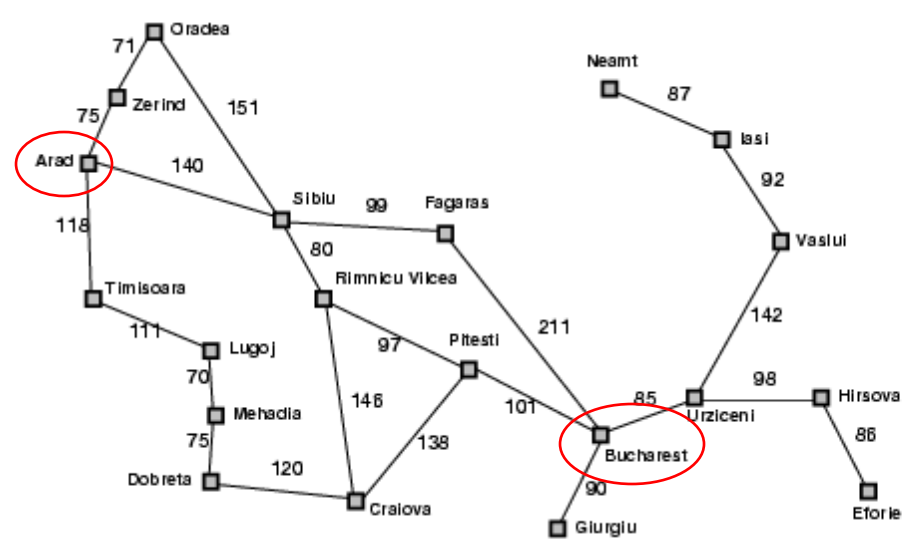
Straight-line distance
to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Greedy Best-first Search: Example

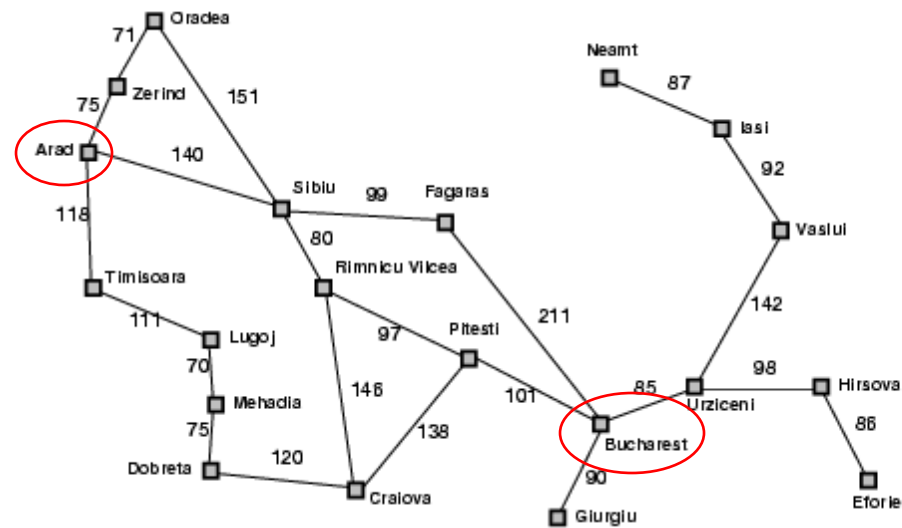
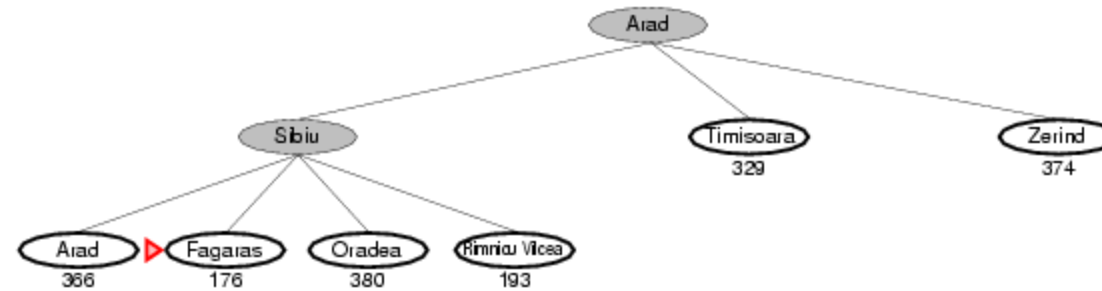


Greedy Best-first Search: Example



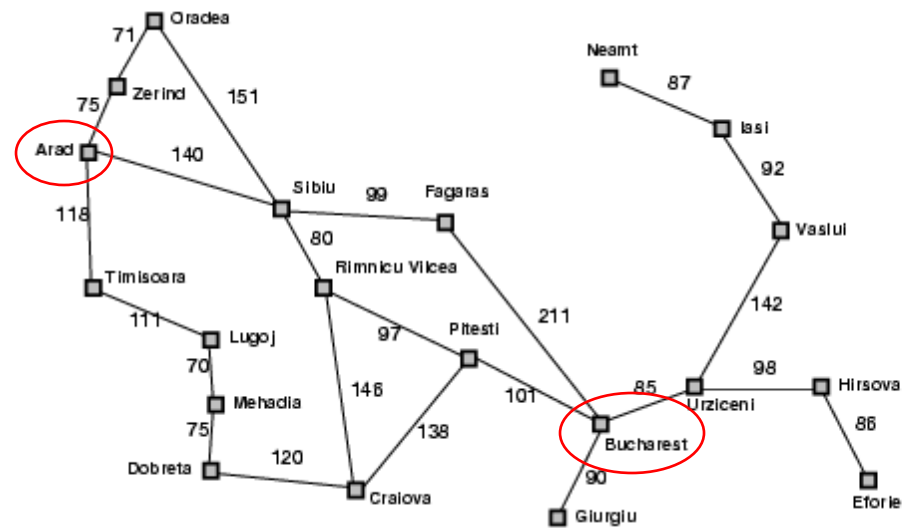
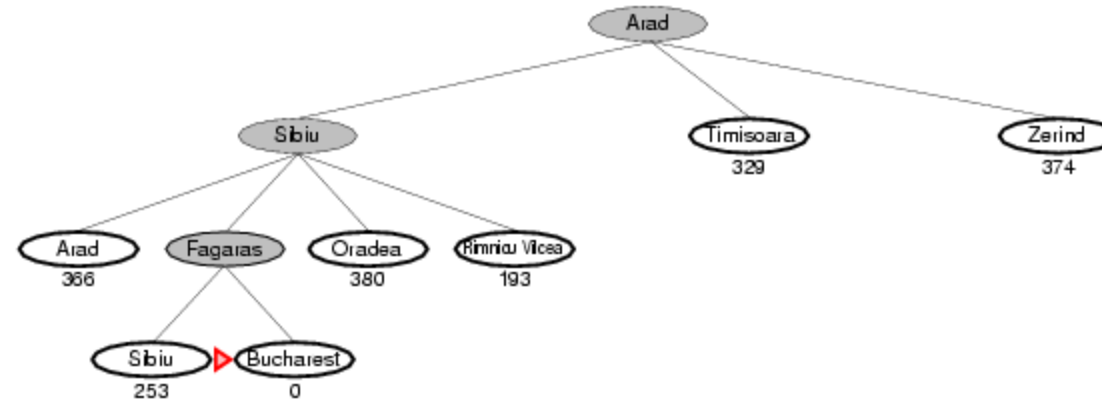
Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Greedy Best-first Search: Example



Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Greedy Best-first Search: Example

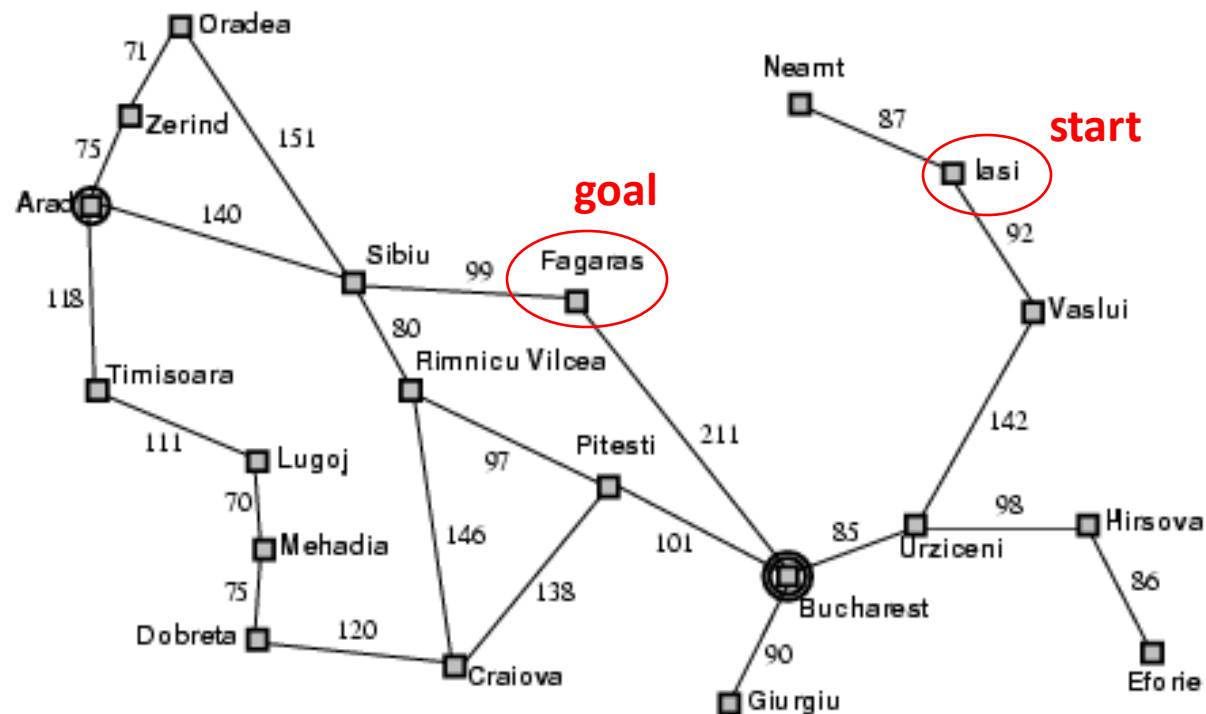


Straight-line distance
to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

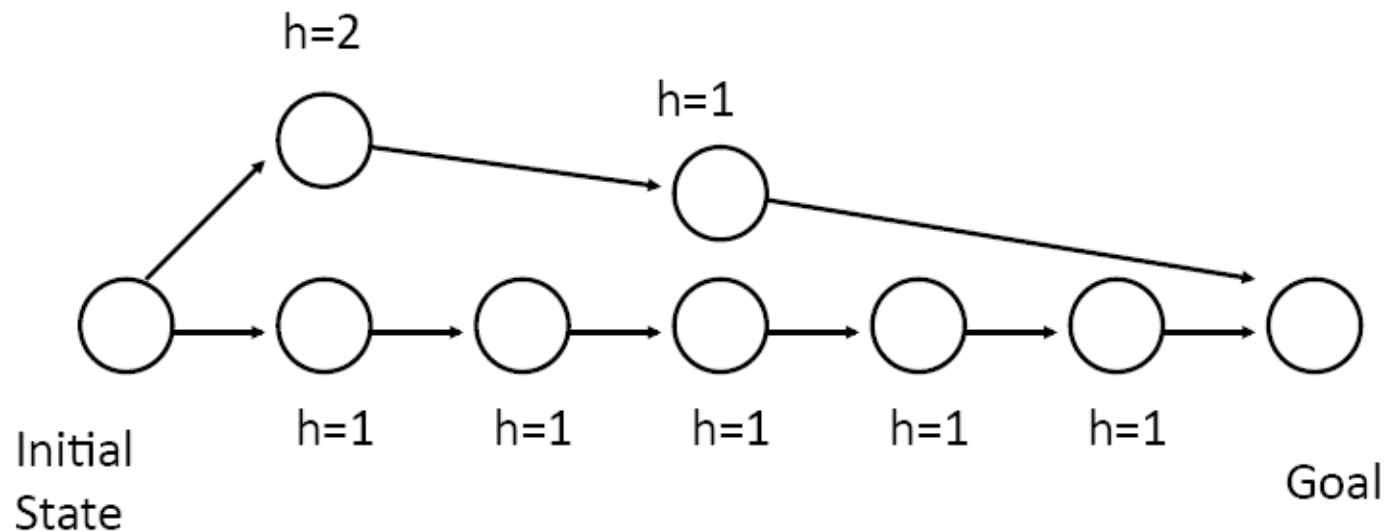
Greedy Best-first Search: Properties

- Complete?
 - No: can get stuck in loops.



Greedy Best-first Search: Properties

- Complete?
 - No – can get stuck in loops
- Optimal?
 - No



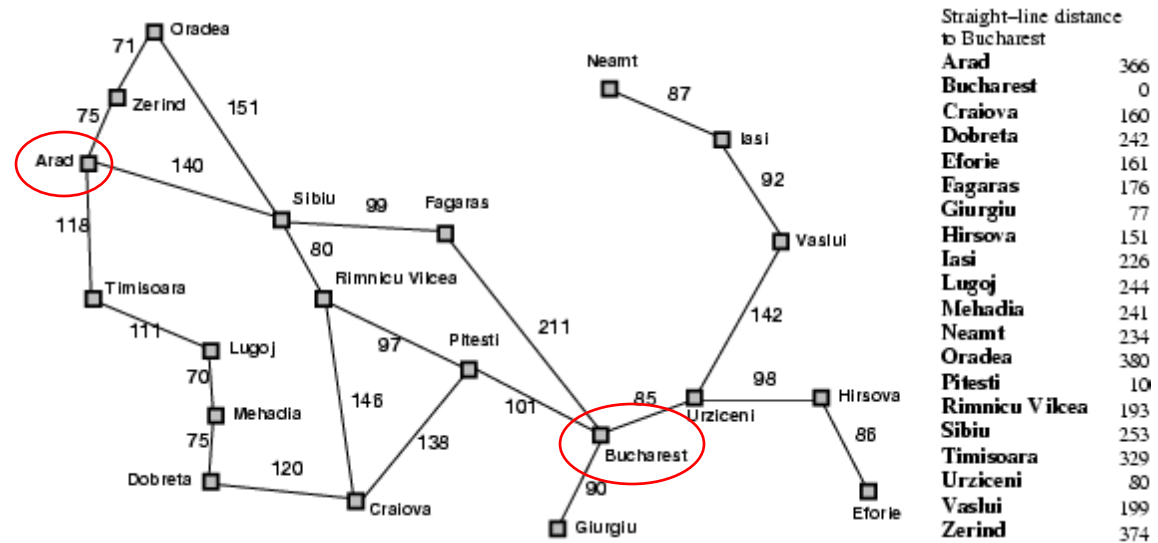
A* Search

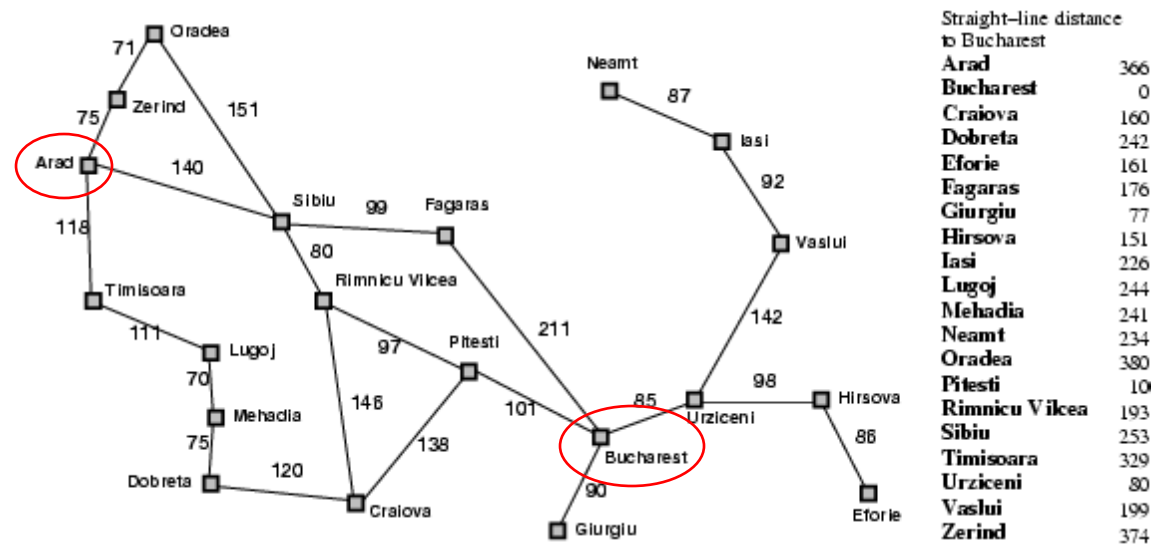
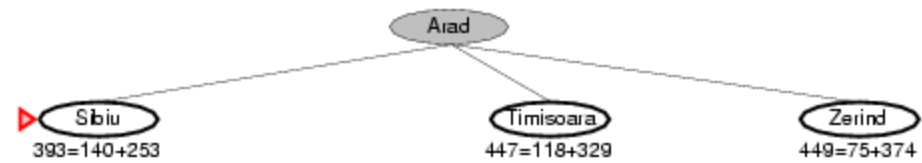
- Idea: Avoid expanding paths that are already expensive.
- Use an **evaluation function**:

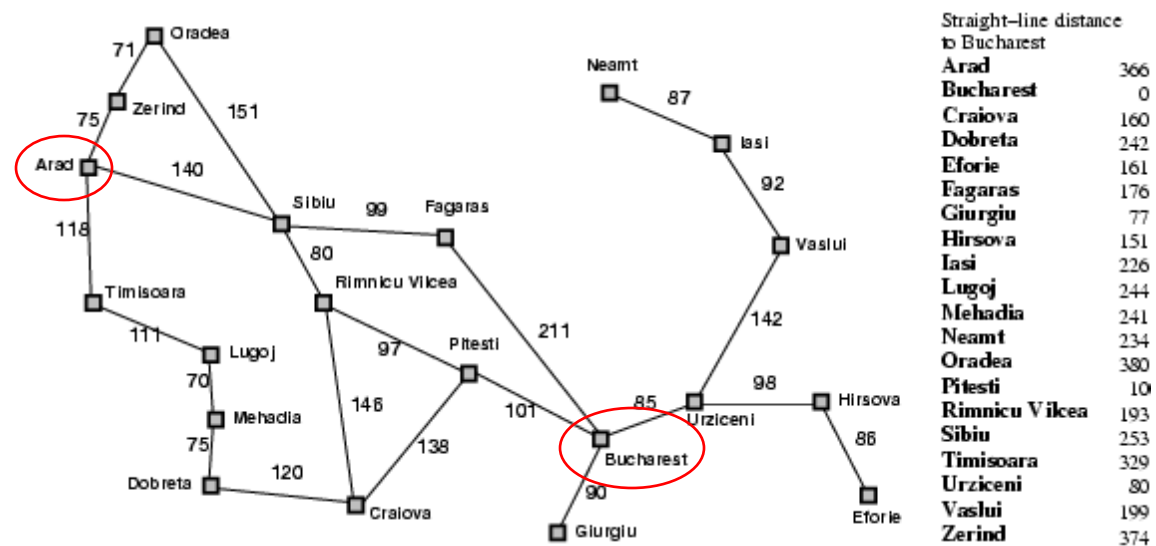
$$f(n) = g(n) + h(n)$$

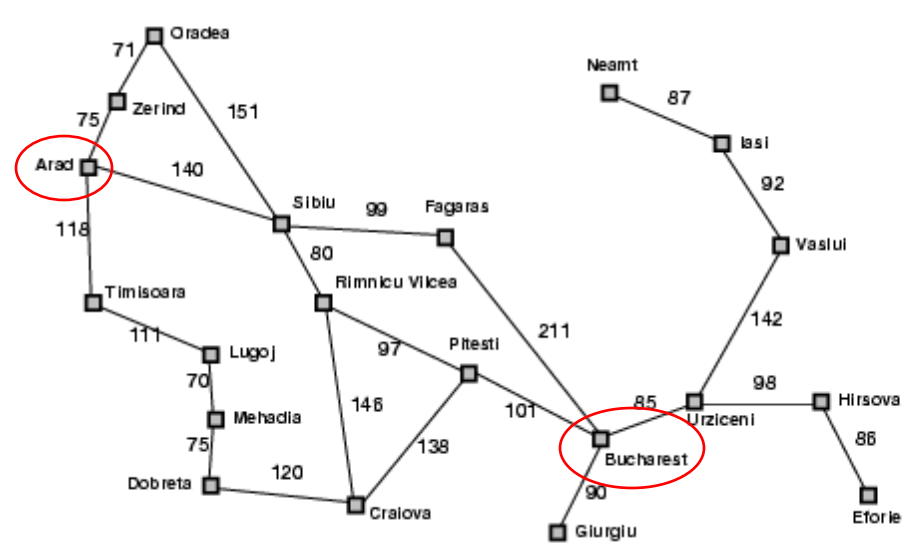
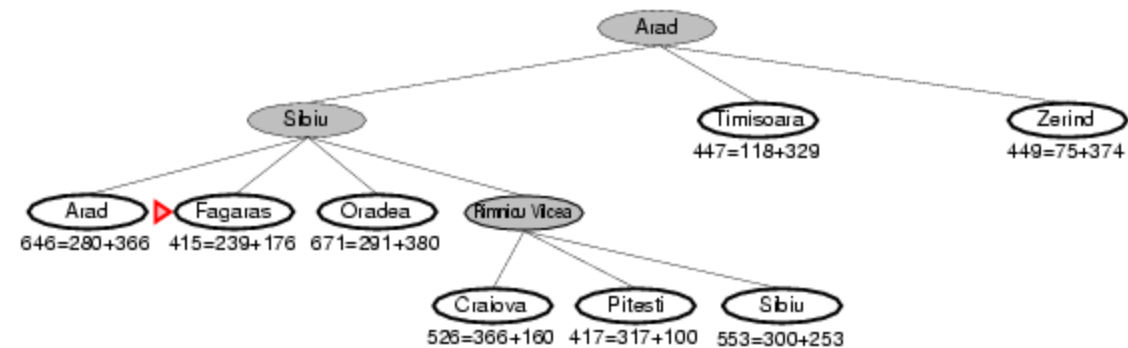
- $g(n)$: actual cost from the start to node n
 - $h(n)$: estimated cost from node n to the goal (heuristic)
- A* expands the node with the lowest total estimated cost $f(n)$, combining both actual cost and heuristic estimate.

Arad
366=0+366



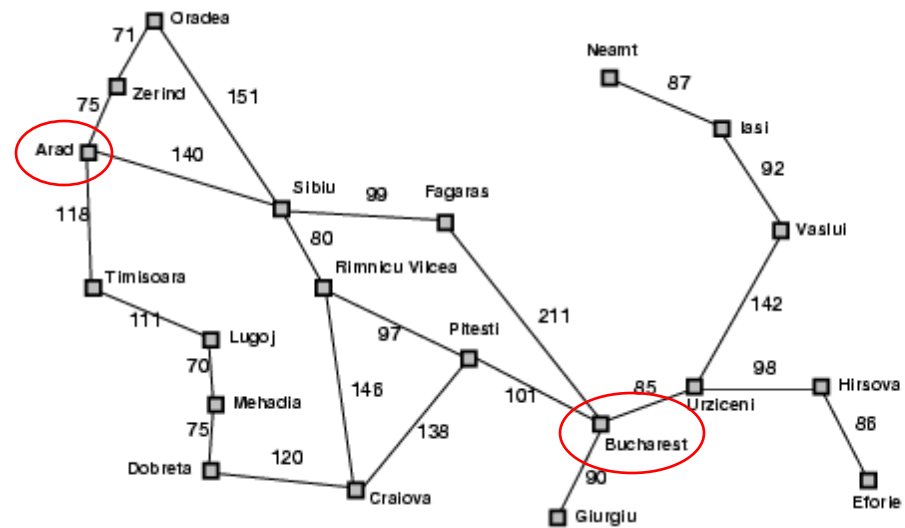
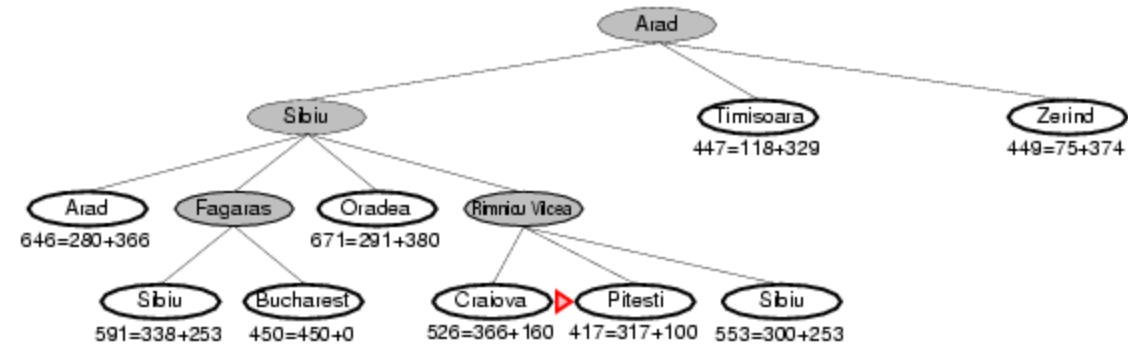






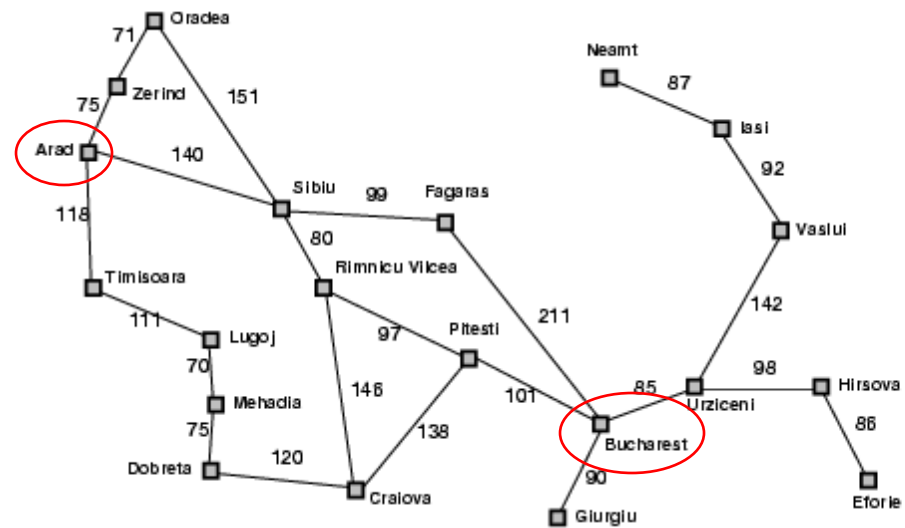
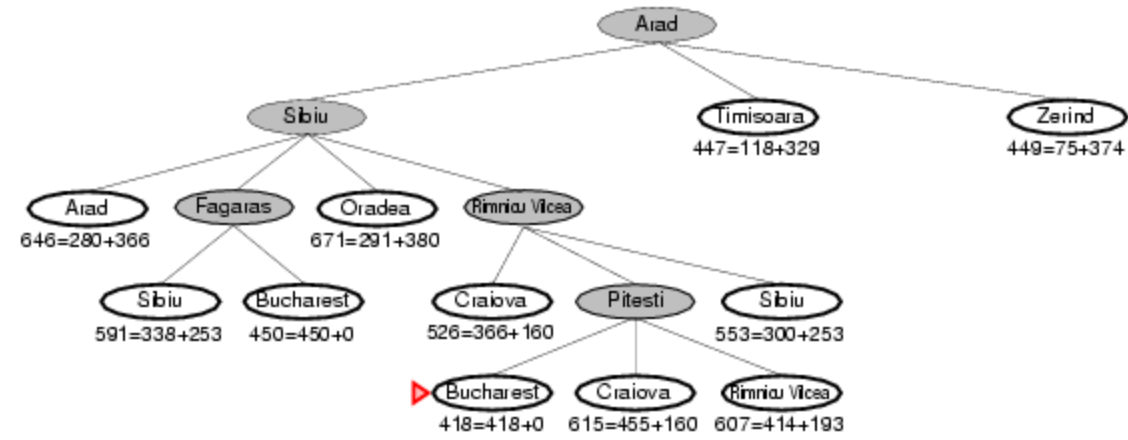
Straight-line distance
to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374



Straight-line distance
to Bucharest

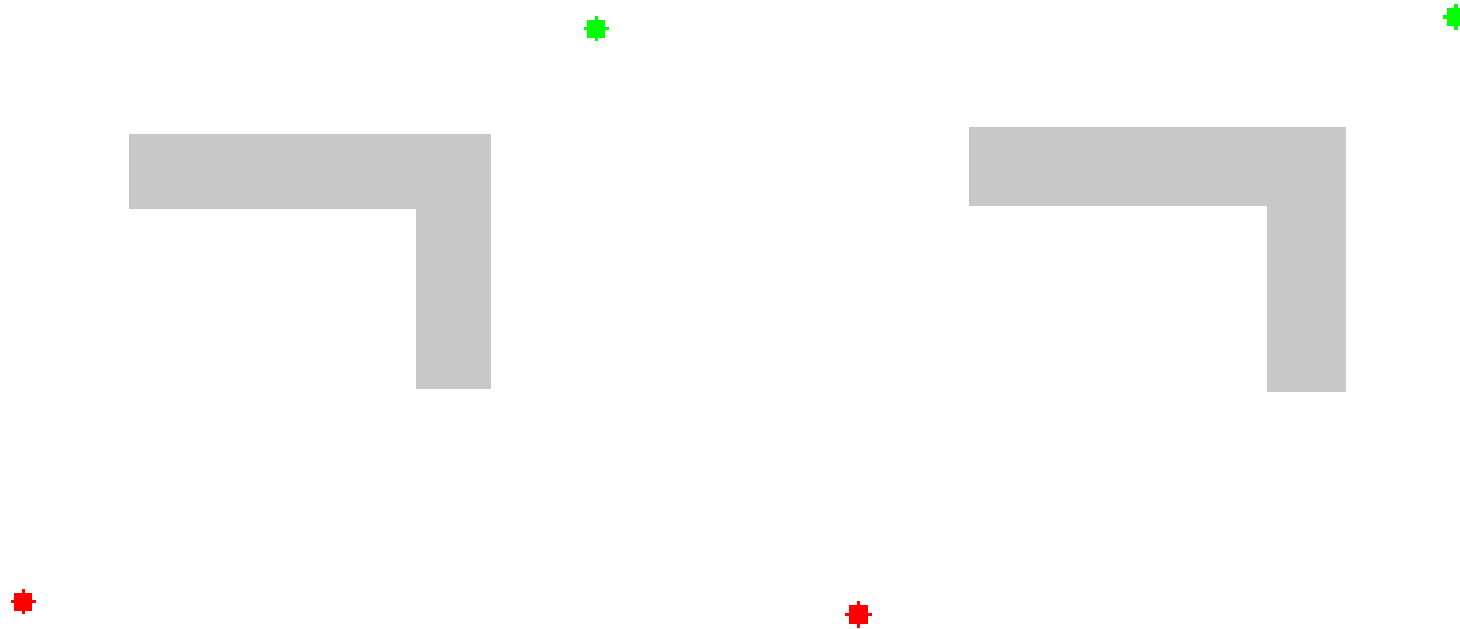
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374



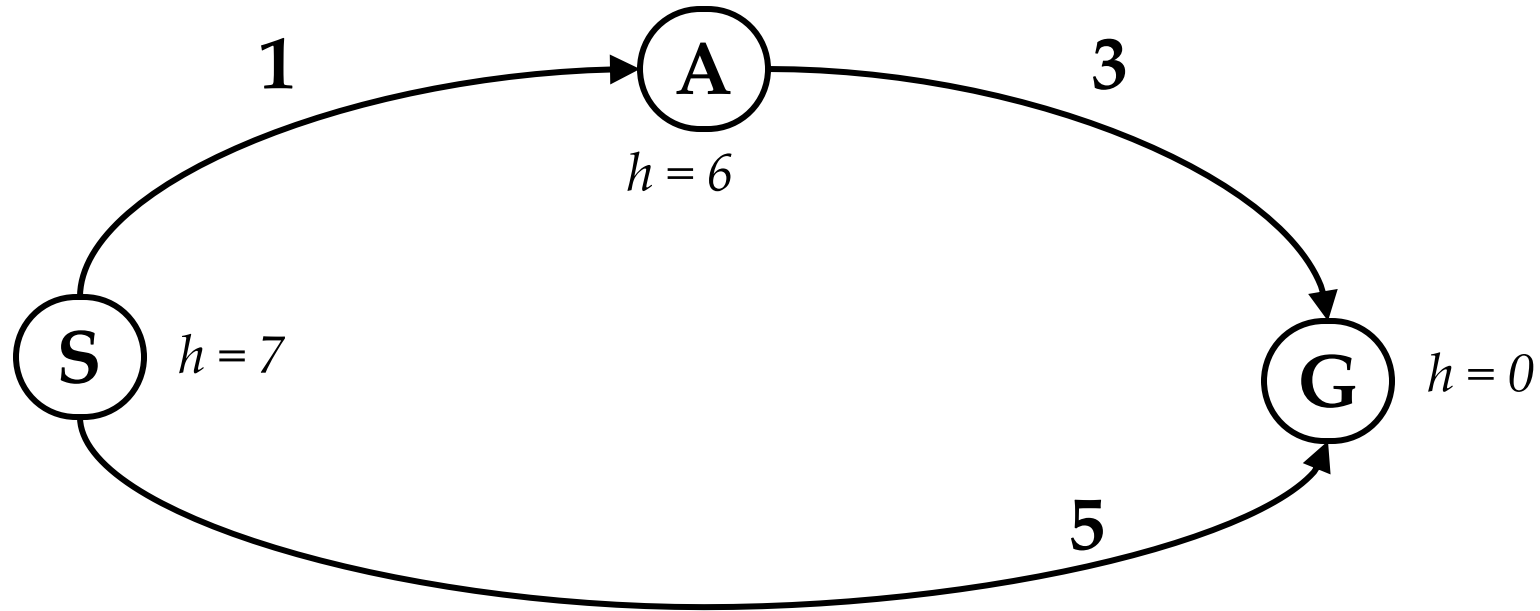
Straight-line distance
to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

UCS vs. A* Search



Is A* Optimal?



	g	h	+
S	0	7	7
S->A	1	6	7
S->G	5	0	5

- What went wrong?
 - Actual goal cost < estimated goal cost
- We need estimates to be less than actual costs!

Admissible Heuristics

- A heuristic $h(n)$ is **admissible** if $h(n) \leq h^*(n)$ for every node n , where $h^*(n)$ is **the true cost** to reach the goal state from n .
- An admissible heuristic is optimistic; it never overestimates the cost to reach the goal.
 - Example: straight line distance never overestimates the actual road distance.
- Theorem: **If $h(n)$ is admissible, A^* search is optimal.**

Designing Heuristic Functions

- Heuristic examples for the 8-puzzle:
 - $h_1(n)$ = number of misplaced tiles;
 - $h_2(n)$ = total Manhattan distance (number of squares from desired location of each tile).

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

$$h_1(\text{start}) = 8$$

$$h_2(\text{start}) = 3+1+2+2+2+3+3+2 = 18$$

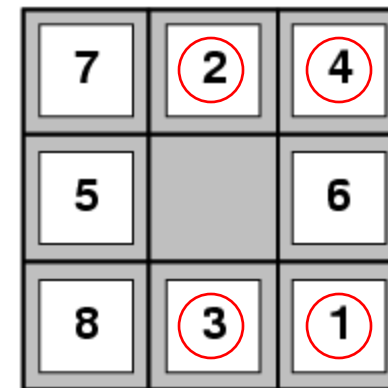
Are h_1 and h_2 admissible?

Heuristics from Relaxed Problems

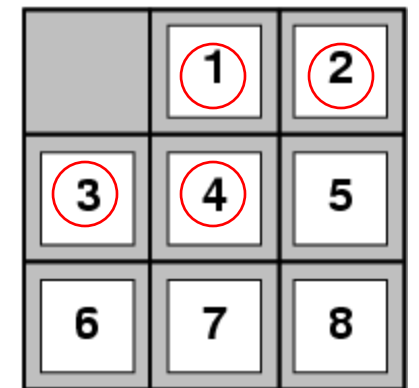
- A **relaxed problem** is one where some of the original constraints on actions are removed.
- The cost of an optimal solution to a relaxed problem provides an **admissible heuristic** for the original problem.
- Example: The 8-Puzzle
 - If tiles are allowed to move to any position, the optimal cost becomes $h_1(n)$
→ the number of misplaced tiles.
 - If tiles are allowed to move to any adjacent square, the optimal cost becomes $h_2(n)$
→ the sum of Manhattan distances.

Heuristics from Sub-problems

- Define a heuristic $h_3(n)$ as the cost of moving a subset of tiles (e.g., tiles 1, 2, 3, 4) into their correct positions.
- To compute $h_3(n)$ efficiently, precompute and store the exact solution cost for every possible configuration of that subset.
- This technique is known as using a **pattern database**, which provides fast and accurate heuristic estimates.



Start State



Goal State

Dominance

- If h_1 and h_2 are both admissible heuristics and $h_2(n) \geq h_1(n)$ for all n , then h_2 **dominates** h_1 .
- Which one is better for search?
 - A* search expands every node with $f(n) < C^*$ or $h(n) < C^* - g(n)$;
 - Therefore, A* search with h_1 will expand more nodes, leading to lower efficiency.

Combining Heuristics

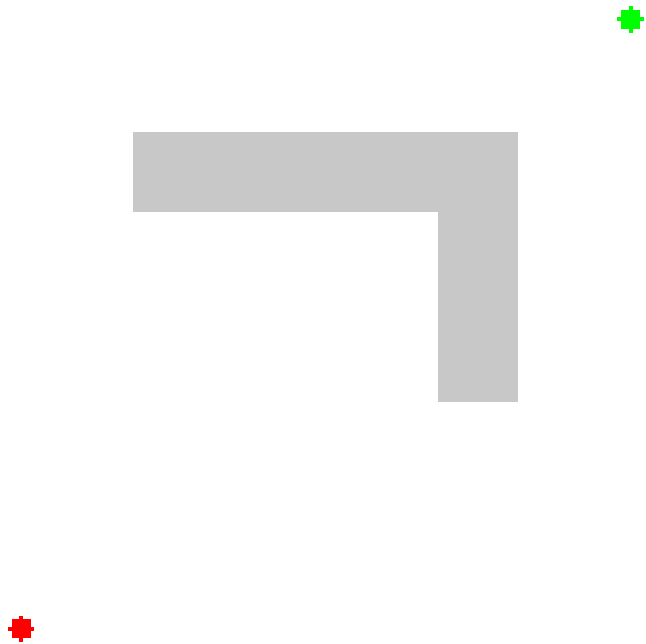
- Suppose we have a collection of admissible heuristics $h_1(n), h_2(n), \dots, h_m(n)$, but none of them dominates the others.
- How can we combine them?

$$h(n) = \max\{h_1(n), h_2(n), \dots, h_m(n)\}$$

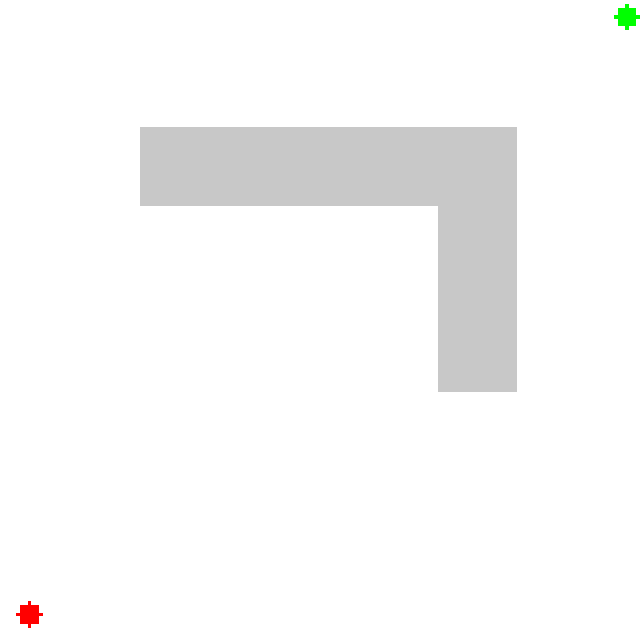
Weighted A* Search

- Idea: speed up search at the expense of optimality.
 - Take an admissible heuristic, “inflate” it by a multiple $\alpha > 1$, and then perform A* search as usual.
 - Fewer nodes tend to get expanded, but the resulting solution may be suboptimal
 - Its cost will be at most α times the cost of the optimal solution.

Example: Weighted A* Search



Heuristic: $5 * \text{Euclidean distance}$
from the goal



Compare: Exact A*

Online Demos

- [Pathfinding \(faramira.com\)](http://faramira.com)
- [Variants of A* for path finding on grids](#)

Algorithm	Complete?	Optimal?	Time complexity	Space complexity
BFS	Yes	If all step costs are equal	$O(b^s)$	$O(b^s)$
DFS	No	No	$O(b^m)$	$O(bm)$
IDS	Yes	If all step costs are equal	$O(b^s)$	$O(bs)$
UCS	Yes	Yes	Number of nodes with $g(n) \leq C^*$	
Greedy	No	No	Worst case: $O(b^m)$ Best case: $O(bs)$	
A*	Yes	Yes (if heuristic is admissible)	Number of nodes with $g(n)+h(n) \leq C^*$	

Summary

- In a **fully observable, deterministic, discrete, and known** environment, we can design a **goal-based rational agent** and use **search** to find solutions.
- What if the environment is more complex?
 - Partial observability → Agent needs internal memory to track hidden state
 - Stochastic dynamics → Agent must plan for uncertainty and contingencies
 - Continuous time or space → Requires real-time control and continuous reasoning
 - Unknown rules or goals → Agent must explore and learn from interaction

Thank You!